



Investigating Integrations of the LLVM Flang Parser with OP2

Samuel Armstrong

Supervisor: Prof. Gihan Mudalige

Department of Computer Science

University of Warwick

September 2026

Abstract

OP2, the Oxford Parallel library for Unstructured mesh solvers, currently relies on Python-based source-to-source translation tools using the `fparser2` library to generate optimised and parallelised code for multi-core and many-core architectures from Fortran source code. This approach suffers from a lack of robustness and standards support, and the performance overhead inherent to interpreted Python. This research investigates the integration of LLVM Flang, a modern C++-based compiler frontend supporting the Fortran 2023 standard, as an alternative to `fparser2` within the OP2 translation pipeline, following the precedent established by OP2-Clang for the C/C++ side of OP2. The work covers the creation of the Flang-based pipeline, including extraction of loop and kernel information, validation of OP2 argument access restrictions, and translation of Fortran kernel subroutines to C++. These stages were successfully implemented, and Flang was integrated as an optional path, with `fparser2` remaining the default. Evaluation found that the new Flang pipeline produced equivalent generated code and performed equivalent validation checks to the existing `fparser2`-based solution on standard applications, while significantly improving Fortran standards support and robustness. Translation performance was mixed, with larger multi-file applications translating substantially faster with Flang, while smaller applications saw modest performance degradation. The Flang pipeline is also limited by practicality and maintainability, as Flang is a heavy dependency with unstable APIs. Future work will focus on exploring alternative parsing approaches, and investigating the trade-offs of introducing semantic analysis to this Flang pipeline.

Contents

1	Introduction	4
2	Background	5
2.1	OP2	5
2.1.1	OP2 Pipeline	5
2.1.2	Translators	5
2.2	Approaches to Parsing	6
2.2.1	Python-Based Parsers	6
2.2.2	Compiler Frontend Parsers	7
2.2.3	Alternative Methods	8
2.3	Precedent	9
2.4	Motivation	9
3	Project Outline	9
3.1	Research Objectives	9
3.2	Stages	10
4	Using LLVM Flang within OP2	10
4.1	Interface Architecture	10
4.2	Parse and Walk Pipeline	11
4.3	OP2 Call Sites	11
4.4	Subprogram Metadata	12
4.5	Kernel Body Structure	12
4.6	Declarations and Statements	13
4.7	Design Decisions and Trade-offs	13
4.7.1	Source Code Slicing	13
4.7.2	Semantic Analysis	13
4.7.3	Parse-tree Layout	14
5	Flang Translation Pipeline	15
5.1	Parsing	16
5.2	Validation	17
5.3	Code Generation	18
5.4	Rewriting the User Driver	19
6	Evaluation	19
6.1	Equivalence	19
6.1.1	Solution Equivalence	19
6.1.2	Validation Equivalence	20
6.2	Robustness	22
6.2.1	Standards Coverage	22
6.2.2	Pipeline Robustness	25
6.2.3	Regressions and Fixes	26
6.3	Performance	27
6.3.1	Translation Performance Breakdown	27
6.3.2	Introducing Batching	28
6.4	Practicality	29

7	Codebase Integration	30
7.1	Implementation Refinement	30
7.2	Setup and Build	30
8	Project Review	31
8.1	Development and Management Methodologies	31
8.2	Outcomes and Timeline	31
8.3	Expected vs Observed Trade-offs	33
8.4	Unforeseen Issues	34
9	Conclusion	34
9.1	Future Work	35
	Note on AI Use	37
	References	38

1 Introduction

Oxford Parallel library for Unstructured mesh solvers, shortened to OP2, is a high-level embedded domain specific language (eDSL) for enabling unstructured mesh applications to be automatically parallelised and executed on multi-core and many-core architectures [1].

Mesh solvers are computational frameworks used to simulate physical phenomena on grids (meshes). Structured meshes follow a predictable pattern in their simulation topology, meaning that the mesh can fit logically into a block of contiguous memory. This reduces the need for complex gather/scatter memory operations and also allows for strong spatial locality (where neighbouring cells can be stored next to each other in memory), making these simulations highly performant, particularly on applications that are memory-bound (primarily bottlenecked by memory rather than compute) [2]. In contrast, unstructured meshes have irregular topologies, where each element can have an arbitrary number of neighbours, and use explicit connectivity information to specify how elements are arranged. This distinction is illustrated in Figure 1. Unstructured meshes are essential for handling certain real-world geometries (like Airfoil [3]), where fitting a regular grid cannot create an accurate representation of the physical system; however, they lead to potentially complex data dependencies. These require complex gathers and scatters where the locations of data cannot be determined at compile time, and can also cause race conditions on that data that require additional synchronisation overhead to manage [4]. This makes unstructured mesh applications more difficult to parallelise and run efficiently.

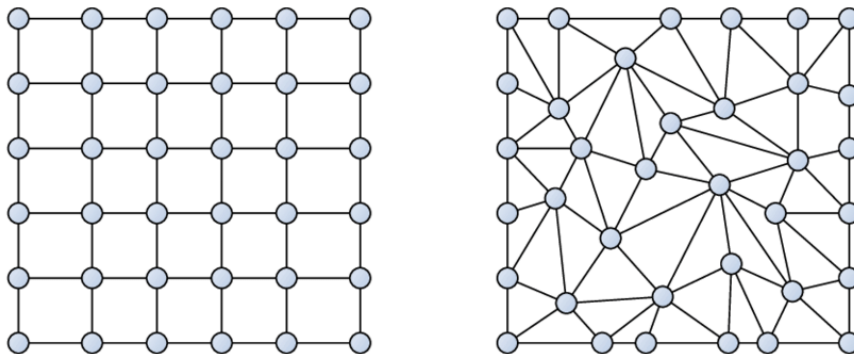


Figure 1: An example of structured (left) and unstructured (right) mesh topologies [5]

OP2 allows users to write a single application using the high-level API in Fortran, C or C++ to describe the mesh and the operations to be performed on it. This describes the application from a purely mathematical and logical perspective, rather than providing precise details for how the application should be parallelised. Before compiling the program, the user uses the Python-based source-to-source translation scripts to automatically convert the generic application they’ve written into low-level code for each target backend that’s supported on their system, generating versions of the code which can be run efficiently on a variety of target hardware and system architectures. This generated code can then be compiled, which links the OP2 runtime functions, creating an optimised executable which can be run on any supported target architecture. Various applications using OP2 have been published [6, 7, 8] that demonstrate strong performance on a wide range of architectures including multi-core CPUs, GPUs, and clusters of CPUs.

For the Fortran side of OP2, the current translation pipeline relies on the `fparser2` Python library [9] to parse the application source code into an AST and extract the necessary information. Its use is a core limitation for the robustness of OP2’s translation pipeline, as it has limited standards support, syntactic checking, and relatively low maintenance. This project aims to

investigate the use of a compiler frontend parser, LLVM Flang [10], as a potential alternative parsing approach that offers more comprehensive coverage across Fortran standards through a compiler-grade grammar, in order to improve the robustness of the translation pipeline.

2 Background

2.1 OP2

2.1.1 OP2 Pipeline

The OP2 pipeline, shown in Figure 2, takes Fortran/C++ source code and converts it to optimised code for each target backend and a sequential version used for debugging [1, 4, 11]. To create the sequential version, the source code is immediately compiled and linked with the OP2 runtime libraries, which represent the precompiled code and public headers in OP2 which are only built once and are used for all applications. This is represented by the right hand branch of the pipeline diagram. The target backend code is created by first passing the source code into OP2’s translator to create the optimised code for each target backend, before completing the same compiling and linking to create the targeted applications [1, 8, 12].

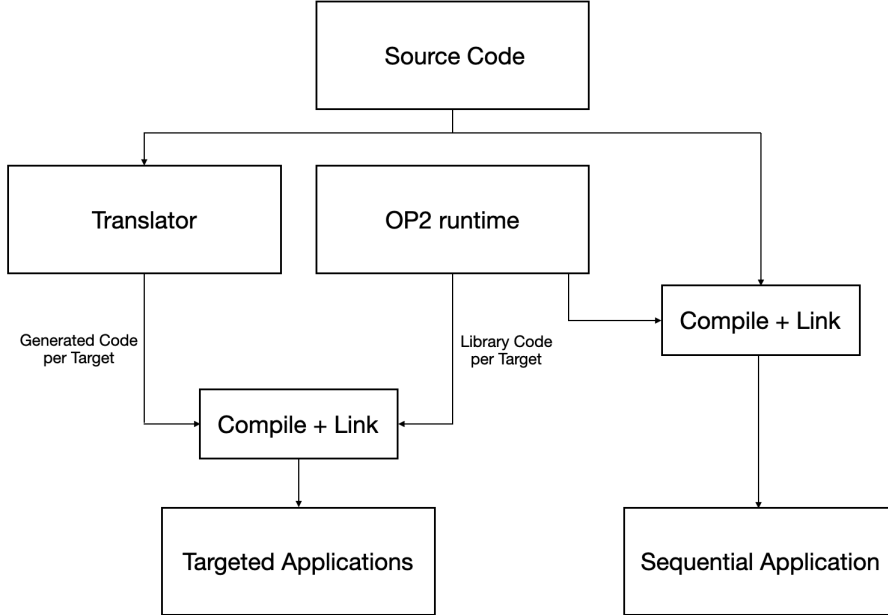


Figure 2: The OP2 build pipeline

2.1.2 Translators

The OP2 translators are responsible for generating the optimised code for each target backend [4, 13]. They operate by first preprocessing and parsing the source code into an Abstract Syntax Tree (AST), then walking it to find the argument information for each `op_par_loop`, and the source code for each kernel, including all functions and subroutines that are called. Kernels and dependencies are then extracted from the parsed AST and are validated to ensure they only perform operations permitted by the specified argument access types and other restrictions.

Once the kernels have been validated, the translator performs the code generation step. This involves rewriting the kernels for restrictions to be run on a device, and rendering Jinja templates containing the optimised code for a specific target. If a kernel fails target code generation, the translator uses the sequential kernel scheme as a fallback, which is used in combination with

successfully optimised kernels in the target application. Next is an optional step of translating the Fortran kernels to C++, used by certain backends (such as CUDA and HIP) so that generated device code can be compiled with the popular C/C++ GPU toolchains, allowing for superior device targeting [1, 11]. Finally, the user driver is rewritten to correctly incorporate the generated kernels, creating the final application for each target backend.

This process is shown in Figure 3.

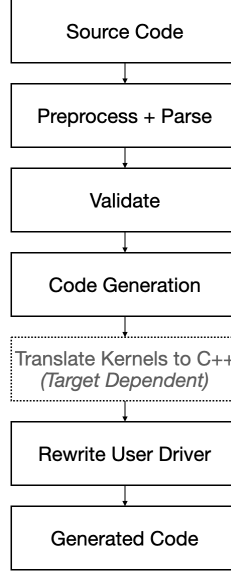


Figure 3: The OP2 translator pipeline

2.2 Approaches to Parsing

One of the key components of the OP2 translators and other source-to-source translation tools is the parser, which is used for extracting key information from the source code, often in the form of an AST. Several types of parsers can be used to achieve this, trading off factors such as speed, robustness, flexibility, and support for language standards.

2.2.1 Python-Based Parsers

Python as a whole has been used in many related domain specific languages and active library frameworks for code generation targeting specific hardware architectures, like OPS [14], Fire-drake [15], OpenSBLI [16] and DeVito [17]. The main issue with using Python in these contexts is that tools written in Python significantly lack the robustness of compiler frameworks such as Clang or GNU [13], as they often have limited syntactic and semantic checking, leading to poor error and warning reporting. This has downstream effects on the maintainability and extensibility of the tools that utilise them, as the code becomes complicated when adding new features or optimisations, making it difficult to debug.

The Fortran parser used in the existing OP2 translators is `fparsr2` [9], which is used to parse the source code into an AST, allowing the program to be easily inspected and manipulated. There are several benefits to using it in the context of OP2 - it can be easily installed via pip as a Python dependency; the AST nodes produced are Python objects, making them easy to interact with; and it can parse source code without requiring the full semantic context of the program, meaning files can be parsed independently of each other.

However, `fparsr2` is limited in its support for Fortran standards, only officially supporting Fortran 2003 and ‘some’ of 2008 [9], meaning it lags behind other parsers in supporting the

latest standard features, as well as older standards that are less commonly used. It also suffers from a relative lack of maintenance, and doesn't use the robust grammar that compiler frontends support. Both of these factors have ramifications for the robustness and reliability of the OP2 translation pipeline. Another key issue with the use of `fparser2` is performance - as a parser that operates purely in Python, it is significantly slower than a compiled C++ parser, which can make the OP2 source-to-source translation process take a long time to complete on large applications.

2.2.2 Compiler Frontend Parsers

A compiler frontend is the part of a compiler that parses the raw source code into an AST, which can be used by the rest of the compiler for lowering into an intermediate representation, such as LLVM IR, which is further used to generate target-specific machine code. Such compiler tooling has been explored in the context of code generation tasks [18, 19, 20] like source-to-source translation, and was applied to the C/C++ side of OP2 in OP2-Clang [13].

One of the modern compiler frontends for Fortran is LLVM Flang [10] - the frontend for the LLVM compiler infrastructure [21], shown in Figure 4. For the initial stage of the compilation process, Flang preprocesses and parses the source code into a parse tree, containing an abstract representation of the Fortran source code, before creating a symbol table through semantic processing, which gives semantic context to the parse tree. Several previous works have explored the use of Flang for the parallelisation of annotated Fortran code [22, 23, 24], demonstrating the use of its parser for translation applications in high performance computing.

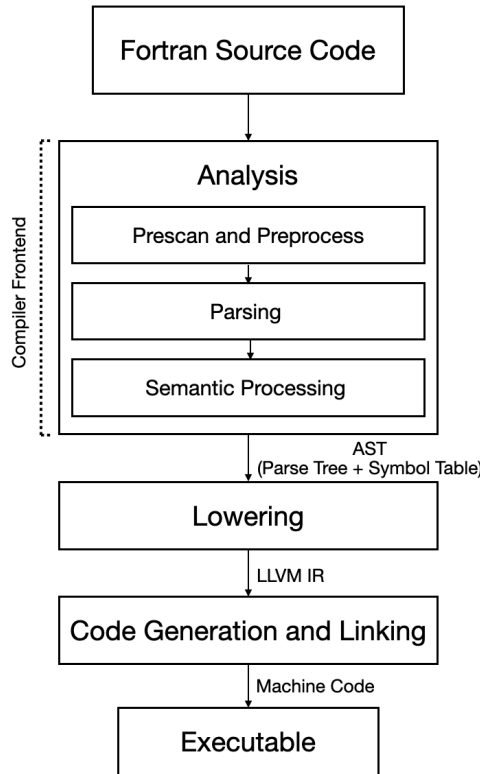


Figure 4: The LLVM Flang compiler pipeline

Flang therefore offers a potential alternative to `fparser2` that could be used for the Fortran side of OP2. There are several advantages to doing this, primarily due to it being a full compiler frontend for Fortran and it being written in C++. As a full compiler frontend, Flang supports up to the latest Fortran 2023 standard [10], and has better support across the board than `fparser2`,

meaning it should be significantly more robust and reliable for use in OP2. In addition, LLVM Flang being written in C++ means it’s compiled into a highly optimised executable, which could make it significantly faster than fparser2 for source code parsing and thereby source-to-source translation.

The potential downsides of using Flang are that it’s a heavy dependency to add to the OP2 codebase; it is a relatively new project, which manifests as stability issues, particularly with its parse-tree API; and it could be difficult to integrate into the existing OP2 codebase due to its lack of native Python bindings.

2.2.3 Alternative Methods

There are also some other options for parsing solutions that could be applied to the OP2 translators, which balance the trade-offs between robustness, practicality, maintainability, and performance in different ways.

One such option is LFortran [25], which is an open-source interactive Fortran compiler built on top of LLVM, somewhat similar to Flang. Unlike Flang, however, LFortran provides Python bindings that allow Fortran to be parsed and the ‘Abstract Semantic Representation’ (ASR) traversed directly from Python. This means the integration into OP2 would be significantly easier than with Flang, as there would be no need to use a C++ interface. LFortran is also a lightweight dependency compared to Flang, as it can easily be installed with conda and doesn’t require the LLVM/MLIR toolchain to be installed. The major downside of LFortran is that it’s still in alpha, meaning it lacks the stability required for use in production or integration into the OP2 codebase - ‘it is expected to not work on third-party code’ [25] - though parsing of ‘Fortran 2018 syntax to AST’ is fully supported at the time of writing.

Another option is Tree-sitter [26, 27], which is a parser generator tool and an incremental parsing library that could be used to parse the Fortran source code. The `tree-sitter-fortran` grammar is widely used, has support for a wide variety of Fortran standards, and has Python bindings that can be easily installed via pip. However, it’s unproven in the HPC domain for translation tasks, and is less robust than Flang, having no official standards-tracking grammar, an incomplete preprocessor and poor line-continuation support. It is therefore strong in terms of practicality, but not a big step up from fparser2 in terms of robustness.

Table 1 summarises estimates of how difficult each parser would be to integrate into OP2; how practical it is from the perspective of the dependency size and installation complexity; how maintainable its integration would be; its robustness on both the tool level and of the resulting OP2 translation pipeline; and its performance. Based on the trade-offs, LLVM Flang was chosen as the best option for use in OP2, as it should offer the biggest gains in robustness, while also potentially bringing significant performance improvements. This, however, comes at the cost of a lack of practicality for users, a difficult integration into the existing OP2 codebase, and subsequently high maintenance burden. For an alternative that better balances these trade-offs, without as much gain in robustness, Tree-sitter is likely the best option. It is far more lightweight and practical for users, and also works natively in Python for an easier integration into the codebase.

Parser	<i>Integration Difficulty</i>	<i>Practicality</i>	<i>Maintainability</i>	<i>Robustness</i>	<i>Performance</i>
fparser2	-	Very Good	Good	Medium	Low
LLVM Flang	Very High	Weak	Medium	Excellent	High
LFortran	High	Good	Good	Low-Medium	High
Tree-sitter	Medium	Good	Medium	Good	Medium

Table 1: Parser trade-offs for use in OP2 source-to-source translation

2.3 Precedent

The precedent for this work has already been laid out by OP2-Clang [13], which addresses a similar problem on the C++ side of OP2 - replacing the use of the Python-based source-to-source translator with one based on Clang LibTooling. This implementation uses LibTooling’s `ASTMatchers` and `Replacements` to modify a skeleton to generate the necessary parallel code, then uses multiple levels of refactoring using LibTooling’s `RefactoringTool` to apply specific optimisations in a ‘flexible, maintainable and extensible manner’ [13]. Two main issues were identified with past works in using source-to-source translation tools to help with writing programs for new hardware [28, 29] that motivated this research - poor extensibility for generating new parallelisations, and the underlying source-to-source translation tool using unreliable or unmaintained software.

This work differs slightly from that being undertaken in this project, as the existing translator already uses an AST-based approach in Python for translating the Fortran source code, whereas OP2-Clang was comparing directly to a version of the translator that did not. However, the overall objective of the research is essentially the same - to replace the use of a purely Python source-to-source translator with one based on a more robust parser.

The evaluation of OP2-Clang was done by comparing the performance of the generated code between the two implementations. The generated code showed near-identical performance to the existing solution (‘difference is less than 2%’ [13]), which was taken to indicate that the code generated by OP2-Clang is likely identical, though this was not rigorously tested and verified. This lack of rigorous evaluation is a key area for improvement from this work, and as such an extensive evaluation of the Flang-based pipeline against the research questions (Section 3.1) is a core objective of this project.

2.4 Motivation

Given the related work discussed, there is a clear gap in the literature for integrating Flang into OP2. Primarily, the use of the LLVM Flang frontend will improve the robustness of OP2, as it is a full compiler frontend for Fortran, meaning it has far better support for Fortran standards and constructs than `fparser2`. The code generated via the Flang-based translation path should also be identical to the code generated by the existing translator on applications that already translate successfully, so the only trade-offs will be surrounding the practicality and maintainability of the integration.

The integration of Flang into OP2 was also identified as a point of future work in the OP2-Clang paper - ‘this work can be extended for OP2’s Fortran API, if a similar infrastructure is developed around the upcoming Flang front end’ [13].

3 Project Outline

3.1 Research Objectives

Given that the key objective was identified as creating a Flang-based translation path that improves the robustness of OP2’s translation pipeline (Section 2.2), the work is organised around four main research questions:

1. Can a Flang-based translation path produce generated code and validation outcomes equivalent to `fparser2` on OP2 applications?
2. Does Flang improve the robustness and standards coverage of the translation pipeline?
3. How does a Flang-based path compare to `fparser2` on translation performance?

4. How do the benefits and trade-offs of a Flang-based translation path weigh up for real users?

Following the implementation, the Flang path is evaluated both quantitatively and qualitatively to address each of these questions. This will thereby assess Flang’s suitability for use in OP2 - and more generally the suitability of compiler frontend parsers for use in domain specific languages for source-to-source translation tasks.

3.2 Stages

At the outset, the project was conceptually split into three stages, with stages 1 and 2 being the core of the project and stage 3 being a research extension. The first stage of the project involves parsing the input Fortran source code with Flang to extract all the necessary information, such as the argument information for each `op_par_loop`, and the source code for each of the kernels with all functions and subroutines that are called. The second stage involves validating that the kernels and loop metadata extracted in stage 1 only perform operations permitted by the specified argument access types and other restrictions. Finally, the third stage involves translating the Fortran source code to C++ using the information extracted with Flang for the relevant target backends, such as `c_cuda`.

As these stages are highly interdependent, and as such implementing them in a modular fashion with separate evaluation of each is infeasible, the implementations were jointly undertaken prior to an overall solution evaluation of the equivalence, robustness, performance and practicality to address the core research questions. The remainder of this dissertation discusses the implementation approach and new translation pipeline; the evaluation results and implications; and the integration of the solution into the OP2 codebase.

4 Using LLVM Flang within OP2

4.1 Interface Architecture

One key issue with using Flang within OP2 is that it doesn’t have a lightweight Python binding that can be used for dealing with the parser, like Clang does with `libclang`. To overcome this limitation, a C++ scanner has been created that can interact with Flang’s C++ parse-tree APIs, collect the necessary information, and emit a JSON document to stdout. A new Python implementation of the translator runs this scanner interface via a subprocess and decodes the resulting JSON document, parsing it into a dictionary that is used to accomplish the translation process. This architecture is illustrated in Figure 5.

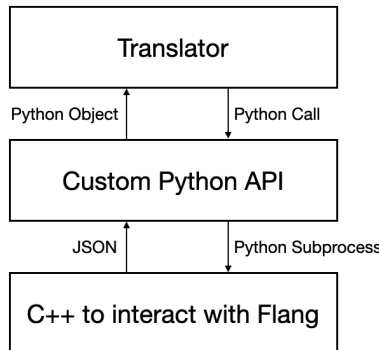


Figure 5: The C++ interface architecture

This C++ component, `op2-flang-scan`, uses LLVM Flang’s parser to read a preprocessed free-form Fortran source file, walk the parse tree, and emit a JSON document describing key informa-

tion about the source code. This includes every `op_par_loop_N(...)` and `op_decl_const(...)` call site along with their argument trees, subprogram metadata for kernel validation, typed declarations and a statement tree.

4.2 Parse and Walk Pipeline

In order for the updated Python translator to rebuild loops, validate kernels, and emit the new Fortran or C++ kernels, it needs a compact representation of each translation unit that is local to the Python process. `op2-flang-scan` therefore runs as a standalone subprocess, reading a single preprocessed Fortran file (in its initial form, batching was later introduced where multiple translation units are processed in a single run - see Section 6.3.2). It uses the LLVM Flang parse-tree APIs to prescan and parse the source code file, before walking the resulting parse tree once with `fp::Walk` to add call site and subprogram events to an events array, and printing the JSON document to stdout. Events are appended in source order, and downstream Python code dispatches depending on each event's `kind`. This pipeline is illustrated in Figure 6.

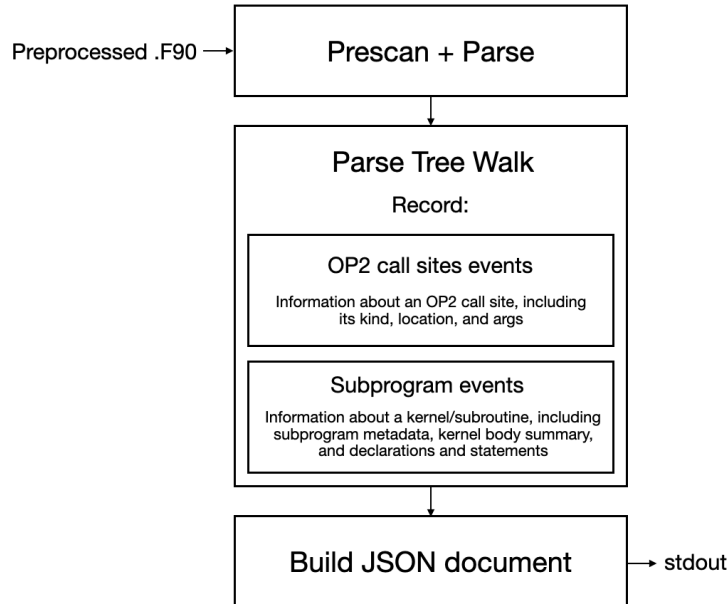


Figure 6: The C++ scanner pipeline (`op2-flang-scan`)

Flang semantic analysis is not used in the scanner, as each translation unit is parsed in isolation so OP2's entity set and access mode checks happen in Python. Materialising Flang's symbol table would not replace this application-wide resolution, while adding additional overhead and complexity to the scanner. This decision is further discussed in Section 4.7.2. Later stages of the Flang compiler, like MLIR lowering and code generation, are also not used - as they are not necessary for information collection. The scanner only accepts free-form source; any fixed-form input is first converted to free-form by the Python driver, using the same `fpp` preprocessing as the `fparser2`-based translation pipeline.

4.3 OP2 Call Sites

To determine the kernels to translate, the scanner records every `op_par_loop_N(...)` and `op_decl_const(...)` call site with its source location and a structured argument tree. Any nested helpers (such as `op_arg_dat(...)`, `op_arg_gbl(...)`, and `op_arg_idx(...)`) become nested `call` nodes, and bare identifiers, folded integer expressions, and character literals become

`name`, `int`, and `string` nodes respectively. Anything the walker cannot decompose into a known node type falls back to a `raw` node carrying the original source text.

These call site arguments are a five-tag tree (`name`, `int`, `string`, `call`, `raw`) which matches the existing `fparser2` design, so the Python translator can rebuild loops and consts without needing the full parse tree. An integer folder handles literals, parentheses, unary plus and minus, and the binary operators `+`, `-`, `*`, `/`, and `**`, including kind suffixes so map indices and `op_decl_const` dimensions remain as integers rather than `raw` text.

Without semantic analysis, Flang cannot tell a derived-type constructor from a call, meaning `op_arg_dat(...)` arrives as a `FunctionReference`. To overcome this, the scanner emits a nested `call` node in the five-tag argument tree and lets the Python side resolve it. The same parse-time ambiguity appears in kernel-body expressions, where parenthesised references such as `q(i)` are emitted as `funcref` nodes rather than `part_ref`.

4.4 Subprogram Metadata

The Python translator also needs entity records for each kernel and dependency, so that dependency closure rewriting doesn't require an `fparser2` AST. Every subroutine and function definition therefore produces a `subroutine_subprogram` or `function_subprogram` event carrying the name, source location, argument list, a `depends` set of reference names seen in the body, and a `source` field sliced from Flang's cooked source stream.

4.5 Kernel Body Structure

In order to perform the same validation checks as the `fparser2`-based validator without walking the AST, each subprogram event also carries a summary of the kernel body. To achieve this, every locally declared array with an array specification is recorded in `locals`, with `dims` listing the identifiers found in its explicit shape bounds. `assignments` is used to record every assignment as `{line, lhs, rhs}` expression trees, and `calls` records every direct `call` with its argument expression trees. The gathered assignments and calls were designed to ignore control flow nesting, as the validators only need to know that a statement exists somewhere in the body, whereas `stmts` (Section 4.6) retains this nesting to allow correct code generation.

Kernel-body expressions are represented by a more comprehensive tree than the call-site arguments, in order to support the additional constructs that are needed for the kernel validation and code generation. They are made up of the following nodes:

- `name`: a bare identifier
- `part_ref`: an array element or section with a named base and subscripts
- `funcref`: a parenthesised `name(...)` that Flang parsed as a function reference
- `triplet`: a section subscript of the form `lower:upper[:stride]`
- `binary`: a binary operator applied to two subexpressions
- `unary`: unary plus, minus, or logical not
- `paren`: a parenthesised subexpression
- `int_lit`: an integer literal, with an optional kind suffix
- `real_lit`: a real literal, with an optional kind suffix
- `logical_lit`: a logical literal

- `char_lit`: a character literal
- `raw`: an undecomposed designator, carrying the original source text
- `unsupported`: any other expression the walker cannot decompose

The kernel validation pattern matches a subset of these (`name`, `part_ref`, `funcref`, `binary`, `paren`, `unary`), while the Fortran-to-C++ code generator walks the full expression tree, including those nodes as well as `triplet`, the literal kinds, `raw`, and `unsupported`. Undecomposed designators such as substring inquiries and structure components become `raw` nodes carrying the original source text, while other constructs the walker cannot decompose (array or structure constructors, defined operators, complex literals, etc.) become `unsupported` nodes, which are opaque and also carry the source text. This allows the Python side to raise clear errors for unsupported constructs, rather than silently misreading them as values.

4.6 Declarations and Statements

For converting Fortran kernels to C++, the key specification constructs (`decls`) and a statement tree that preserves control flow structure (`stmts`) are also needed. To collect `decls`, the scanner records the typed declarations (with type, dimensions, parameter attribute, entities and initialisers), `PARAMETER` statements, and `DATA` statements. External declarations like `USE`, `IMPLICIT`, and `EXTERNAL` are not added to the `decls` tree as nodes.

`stmts` is built as a nested tree of constructs supported by the existing translation pipeline, which preserves `if/do` nesting to allow the C++ code generation to emit the same control flow as the original Fortran source code. It includes nodes for assignments, calls, control flow and loops, and a small set of simple statements (`return`, `stop`, etc.). Constructs such as `DO CONCURRENT` and labelled `DO` become `unsupported` nodes, matching the same gaps as the `fparser2` C++ path. Function subprograms also carry `result_name` and `result_type` from the function statement's `RESULT` clause and prefix type, which the downstream Python-based rewriter uses when mapping Fortran function results onto C++ return values.

4.7 Design Decisions and Trade-offs

4.7.1 Source Code Slicing

One of the key design decisions was to slice kernel bodies from Flang's cooked source code stream rather than using an `Unparse`, so the kernels can be rewritten from text alone. This was possible because the cooked stream is already free-form, lowercased, comment-stripped, and `INCLUDE`-expanded, and can therefore be used directly by `flang_writer` to rewrite kernels without needing an AST. This decision was made as it avoids depending on Flang's `Unparse` templates, which could introduce breaking changes across LLVM releases due to the API instability, and therefore reduces the maintenance burden of this stage of the pipeline.

4.7.2 Semantic Analysis

Another key design decision was whether to include Flang's semantic analysis in the scanner pipeline, to supplement the parsing. There would be several benefits to including it, firstly, a Flang symbol table would let the scanner decide whether `q(i)` is an array element or a function call, and whether `op_arg_dat(...)` is a derived-type constructor or a call. Without it, kernel-body expressions emit `funcref` nodes that Python later reclassifies against known `Function` entities anywhere in the application, and call-site helpers such as `op_arg_dat(...)` become nested `call` nodes in the five-tag argument tree. This would mean that a local array that shares a name with a kernel elsewhere would not be misclassified as a call, and the `depends` list

would name actual callees rather than every `CallStmt` and `FunctionReference` in the body, including intrinsics, array references, and dummy arguments. In addition, `USE` association and `IMPLICIT` typing would make imported types, parameters, and procedures (as well as implicitly typed locals) visible with real types instead of never being recorded in `decls`. The rank and type of subscripts would also be known, so code generation would not have to infer the form of each reference (scalar, array element, or section) from the node shape. Fortran type errors could then be reported at translation time, rather than falling through as silent `raw` nodes.

There are, however, several significant downsides to using semantic analysis. Flang’s semantic analysis lives behind a larger link surface than the parser libraries (`FortranSemantics` and a `SemanticsContext`, with intrinsic tables and module search paths), so including it would enlarge the dependency. In addition, parsing with Flang can only be performed on one file (translation unit) per call, while its semantic analysis expects a compile job per unit, including `.mod` files that need to be produced by analysing those modules first. Converting the batch of files from a multi-file application into this pipeline would require topological ordering, emitting application and OP2 `.mod` files, and feeding them back into later units. This is essentially equivalent to creating a full Fortran compiler driver, and would add a lot of complexity, overhead, and maintenance burden to the pipeline. Another potential shortcoming is that OP2 kernels can also call helpers defined in other files that the translator records as `Function` entities rather than as used module symbols, so a per-file Flang symbol table could still miss that closure even after every file had been analysed.

The decision was therefore made to omit Flang’s semantic analysis in the scanner pipeline, to keep the Flang path as lightweight, performant, and maintainable as possible. Not using it means no Flang symbol table is built, with the trade-off being that resolution work is performed on the Python side. Therefore, as discussed in Section 4.5, argument expression trees may use a `raw` node when an expression cannot be decomposed, and parenthesised references are often emitted as function-like nodes that are later resolved. When building the dependency graph, Python keeps a name if it resolves to a known `Function` entity or appears as a direct `CALL`; unresolved `FunctionReference` names (array references) are dropped, mirroring the `fparser2` `Part_Ref` handling, while unresolved `CALL` names are retained so unknown callees can still be reported.

4.7.3 Parse-tree Layout

A third design decision was which parse-tree layout to use in the scanner, and therefore which Flang (LLVM) versions to support. Flang represents most parse-tree nodes using three conventions: wrappers, tuples, and discriminated unions [30], with tuple nodes implemented by the `TUPLE_CLASS_BOILERPLATE` macro and a `TupleTrait` that exposes a named `std::tuple` member whose elements are accessed by type rather than field name. This tuple-trait layout has been present since Flang’s first LLVM release, but some common nodes were exceptions to this convention up until LLVM 23, instead listing named public members. LLVM 23 resolved this discrepancy by applying a parse-tree trait conversion that moved most of those remaining ‘named member’ nodes onto the tuple-trait layout [31, 32]. The choice was therefore whether to target the updated tuple-trait layout used from LLVM 23 onward, the older named member fields present on $\text{LLVM} \leq 22$, or attempt support for both layouts. The different parse-tree node layouts across versions are summarised in Table 2.

LLVM version	Layout	Notes
≤ 22	Named members	Named members used on walker nodes (<code>.call</code> , <code>.subscripts</code> , etc.)
23	Updated tuple-trait	Applied the parse-tree trait conversion
24 (unreleased)	Updated tuple-trait	Snapshot used during development (built from source, tracking <code>main</code>)

Table 2: LLVM Flang parse-tree node layouts across versions

As part of the tuple-trait conversion, frequently used nodes gained regular accessor methods such as `Base()` and `Subscripts()`, while others are read with `std::get`, which is more uniform than the previous mix of public data members. This would make the walker (within the scanner) easier to both write and extend when new statement or expression kinds are added. In addition, targeting the tuple-trait layout would keep the walker aligned with the main branch of LLVM, as later releases do not keep the named member fields (removed in LLVM 23), so that layout exists only on LLVM ≤ 22 . This alignment would reduce the maintenance burden of the walker, as bug fixes and features introduced by new releases could be absorbed without a complete rewrite. Conversely, if the older named member fields were used, the scanner would not compile against LLVM 23 or later at all, and a rewrite would still ultimately be required once those older versions are no longer shipped or supported. This makes the updated tuple-trait layout the more logical design choice for the scanner.

There are, however, significant costs to solely targeting the updated tuple-trait layout. On the nodes the walker uses, LLVM ≤ 22 expose only the older named members and would thus not be compatible with the scanner if it only targeted the converted layout. This is an issue for practicality, as, for example, Ubuntu 24.04’s archive packages currently stop at `libflang-20-dev` [33], so obtaining a working version of LLVM Flang would become more cumbersome (for example, by building from source).

The alternative would be dual-supporting both layouts, which would resolve both the issue of compatibility with older versions and long-term support. The problem with such a solution is that it would mean duplicating every node access in the walker, making it far more complicated to create. Additionally, it would commit the project to maintaining a compatibility layer for the named member API that Flang has already effectively replaced.

Given these trade-offs, the decision was made to solely target the converted tuple-trait layout, in order to keep the walker as small and maintainable as possible, while aligned with the current version of Flang. For example, the walker uses `CallStmt::t` rather than the older named member `cs.call`, and `ArrayElement::Subscripts()` and `Base()` rather than `.subscripts` and `.base`. This decision also means that small signature changes can be absorbed without rewriting the walk, for example a `_has_include` guard already handles a change to `Parsing::Parse` on LLVM 24 (tracking `main`, not a released version at the time of writing), so both LLVM 23 and later snapshots are supported.

The key downside of this decision is that it causes the version floor for the scanner to be LLVM Flang ≥ 23 , which makes the pipeline slightly less practical, but avoids the complexity of dual-supporting both layouts. Nevertheless, the nature of Flang’s parse-tree API being unstable means there is a risk of future breaking changes, imposing future maintenance costs which cannot be avoided.

5 Flang Translation Pipeline

Several Python modules have been created to form the Flang translation pipeline, which use the information from the C++ scan to run the translation process. The key translation steps of preprocessing, information parsing, validation, code generation, and user driver rewriting are

the same as the existing pipeline, but work off the information provided by `op2-flang-scan`, highlighted in Figure 7.

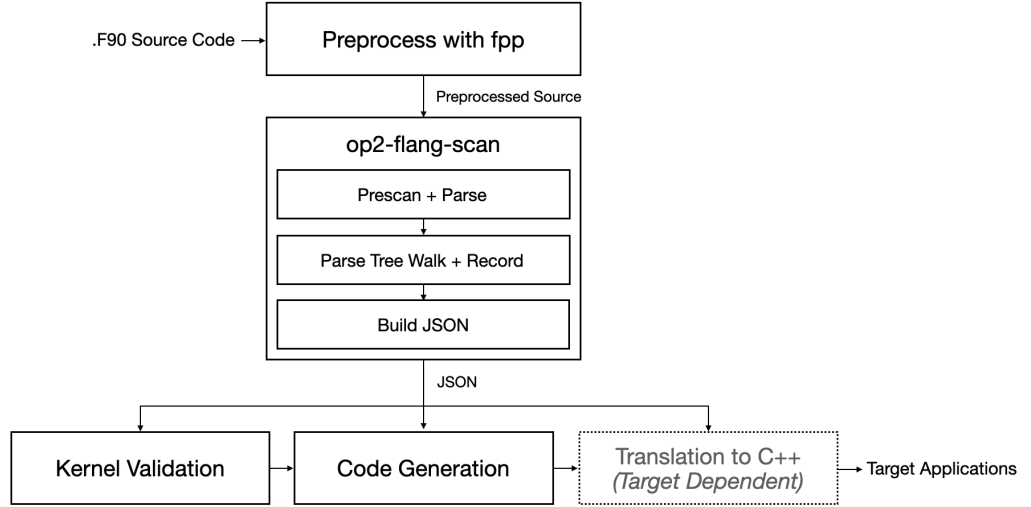


Figure 7: The updated translation pipeline

5.1 Parsing

The initial parsing step passes the preprocessed source code to the `op2-flang-scan` C++ binary as a subprocess, where it uses LLVM Flang to emit a JSON containing the information discussed, including the OP2 call site and subprogram events. This JSON is used by `flang_parser.py` to materialise a `Program` object containing the loop and const call sites in `program.loops` and `program.consts`. Each subroutine or function event becomes a `Function` in `program.entities`, carrying `flang_source` (the Fortran source code slice from the subprogram metadata), `flang_body` (the locals, assignments, and calls trees from the kernel-body structure), and a raw `depends` list of callee names.

`flang_body` also stores the data used for Fortran-to-C++ translation, made up of the `decls` and `stmts` trees, described in Section 4.6, together with `is_function`, `result_name`, and `result_type` on function subprograms.

At this stage no `fparser2` AST is attached, so `program.ast` and each `entity.ast` are set to `None`. If `op2-flang-scan` fails on a file, the driver prints a warning and instead falls back to the `fparser2` parsing pipeline on the same preprocessed source for the file in question, which attaches the ASTs and marks the file as `used_parser = "fparser2"`. Allowing mixed parsing backends via this fallback system, with some files parsed by Flang and others by `fparser2`, avoids the use of Flang introducing any regressions to the existing pipeline. However, a loop can only be validated on the Flang path if the kernel and every transitive dependency carry `flang_body`, so if any entity in the closure came from an `fparser2` fallback, validation and code generation fall back to `fparser2` for that loop too. This requires lazily attaching an AST to every Flang-parsed program in the application, to ensure helpers from other Flang-parsed files are visible to the kernel that required fallback, though kernels that did not require fallback will still validate on the Flang path without requiring the attached ASTs. Nevertheless, Section 6 shows that Flang parsing works successfully in almost all cases, so this fallback is implemented purely to deal with worst case scenarios.

Before loop validation can proceed, each kernel’s dependency closure must be resolved. Under the `fparser2` path this happens per program immediately before validation;

`parseFunctionDependencies` walks `program.ast` and resolves `Part_Ref` nodes against known

Function entities. The Flang path performs an equivalent step once per application, where the raw **depends** names recorded in the JSON subprogram events are resolved against known **Function** entities across all programs in the application, using the filtering rules described in Section 4.7.2. This allows multi-file kernels to be resolved without a Flang symbol table.

5.2 Validation

The next step in the translation pipeline is kernel validation. For each (**loop**, **program**) pair in `app.loops()`, the driver checks that the loop's kernel and every dependency can be validated using the available information - meaning all are **Function** entities with `flang_body` set. If this is not the case, the driver falls back to `fpars2` by lazily attaching `fpars2` ASTs to all Flang-parsed programs in the application, then using the original `validateLoop`, which walks `entity.ast`.

The validator recurses through child subroutine calls via the call graph, using the `calls` entries in `flang_body` together with `func_ref` and `part_ref` nodes in assignments. When `loop.fallback` is set it signals that an optimised target-specific kernel should not be generated for that loop (for example, because of an incompatible stride or increment form), with the sequential kernel emitted instead to ensure correctness. The checks performed are conceptually the same when using Flang as `fpars2`, matching the functionality of the existing validator, and involve the following:

- **Kernel resolution** $\rightarrow$ `OpError`
 - missing kernel: no subroutine or function named `loop.kernel`
 - ambiguous kernel: more than one kernel name match
 - arity mismatch: `len(loop.args)` does not match the kernel argument count
 - duplicate entity names: the same entity name appears multiple times in the closure
- **Unknown dependencies** $\rightarrow$ `loop.fallback = True`
 - there are names in the dependency closure that do not resolve to a known entity
- **Parameter/const name conflicts** $\rightarrow$ warning
 - a parameter name is also an OP2 const
- **Const writes** $\rightarrow$ `loop.fallback = True`
 - an OP2 const that is not a dummy parameter must not appear on the left-hand side (LHS) of any assignment (no constants are written to)
- **Slice incompatible with stride** $\rightarrow$ `loop.fallback = True`
 - a slice or section is incompatible with stride insertion
- **Read-only arguments written** $\rightarrow$ `loop.fallback = True`
 - arguments declared `OP_READ` or passed via `op_arg_idx(...)` must not appear on the LHS of any assignment (read-only arguments are not written to)
- **Improper OP_INC increments** $\rightarrow$ `loop.fallback = True`
 - the parameter must be used as a correct increment of the form `x = x + delta` or `x = x - delta`

- the right-hand side (RHS) must reference the parameter exactly once, at the same index as the LHS
- **Runtime-dimension local arrays** → warning
 - locally declared arrays (from `flang_body.locals`) whose shape bounds reference a kernel parameter or OP2 const are flagged
 - these imply arrays sized at runtime, which can be problematic (especially on GPU)

As parenthesised references are often emitted as `funcref` rather than `part_ref` (Section 4.7.2), the checks treat `part_ref` and `funcref` as equivalent whenever the base name is a kernel dummy. A dummy argument cannot simultaneously be a subprogram, so comparing the name against known **Function** entities is sufficient for OP2’s access mode validation without Flang types.

5.3 Code Generation

The code generation step in the Flang translation pipeline iterates over the target schemes (sequential, CUDA, HIP, etc.), for each looping over the kernel entities to generate the source code for the target-specific kernel. If a scheme’s required Flang metadata is missing (`flang_source` for Fortran emission, `flang_body` for C++), that scheme falls back to the `fparser2` path. For each kernel, this step performs some of the following operations depending on the specific target scheme:

- **Constant renaming** (`flang_writer.rename_consts`)
 - Replace OP2 const names everywhere except in string literals and where the name is also a dummy argument of that entity
- **Function renaming** (`flang_writer.rename_function_definition`)
 - Rename the function definitions and matching end statements
- **Function call renaming** (`flang_writer.rename_function_calls`)
 - Rewrite function call references to correspond to the renamed functions
- **Entity renaming** (`flang_writer.rename_entities`)
 - Rename a group of subprograms (definitions and cross-calls) for OpenMP SIMD variants
- **Externals removal** (`flang_writer.remove externals`)
 - Strip external declarations from the source code
- **Fixing Hydra I/O** (`flang_writer.fix_hydra_io`)
 - Replace `write(...)` statements with `continue` and certain Hydra helper calls with `stop`
- **Stride insertion** (`flang_writer.insert_strides`)
 - Rewrite indexed accesses of matching loop arguments to apply a stride
- **Atomic increment insertion** (`flang_writer.insert_atomic_incs`)
 - Rewrite increments to `op2_ret = atomicAdd(...)` and declare `integer(4) :: op2_ret`

- **Source emission** (`flang_writer.write_source`)
 - Concatenate entity bodies in dependency order and wrap overlong lines with Fortran continuations

For backends that translate Fortran kernels to C++, such as `c_cuda` and `c_hip`, each kernel and its transitive dependencies are translated using `flang_kernels.py`. This first applies Hydra I/O cleanup and, for CUDA/HIP, `const` renaming and atomic-increment insertion (`call atomicAdd(...)`) on the nested `decls/stmts` trees from the JSON. `flang_kernels_c.py` then resolves types and parameter accesses from those trees (together with the `op_decl_const` metadata) and walks the statement tree to emit the C++ kernels. To build the type information necessary for the C++ translation, the Flang path builds `info.consts` directly from `Application.consts()` (the `op_decl_const` calls that have already been parsed), avoiding the need for a second parse.

The host wrappers and master kernel modules are rendered from the same style of Jinja templates as the existing translator, with each scheme’s `translateKernel` extended to include a Flang branch, while the scheme and template architecture remain the same. If the scheme cannot generate a loop-host wrapper (or `loop.fallback == True`), target-specific kernel generation is skipped and only the sequential fallback scheme is emitted.

5.4 Rewriting the User Driver

The final pipeline step rewrites the user’s driver program to call the generated target kernels instead of the original `op_par_loop` macros. Under the `fparser2` path this is done using `translateProgram`, which walks `program.ast`, or the regex-based `translateProgram2`. The Flang path always uses `translateProgram2`, performing a regex/text rewrite on `program.source` without requiring the `fparser2` AST.

6 Evaluation

The evaluation was conducted using a from-source build of LLVM Flang tracking `main` and the latest version of `fparser2` at the time of writing, 0.2.4. The applications and tests were run via WSL on an NVIDIA RTX 3080 GPU (where the `c_cuda` backend was used).

6.1 Equivalence

6.1.1 Solution Equivalence

To assess the ability of the Flang translation pipeline to generate solutions that were equivalent to those produced by the `fparser2` pipeline, from the same source code, a small test suite of three example applications was used. The examples used were `airfoil`, simulating fluid flow around an airfoil; `mesh_res`, simulating diffusion on an unstructured triangle mesh; and `tri_diff`, a triangle mesh updated using both geometric and flux residual updates.

For initial evidence of equivalence, each of these applications was translated to the following OP2 target backends [34] using both the Flang and `fparser2` pipelines:

- `genseq`: Code-generated sequential build
- `openmp`: Multi-threaded CPU build using OpenMP
- `cuda`: NVIDIA GPU build using CUDA
- `c_cuda`: NVIDIA GPU build using CUDA with JIT compilation

- `f_openmp`: OpenMP multi-threaded Fortran build
- `f_cuda`: Native CUDA Fortran build
- `f_c_cuda`: Fortran interop with JIT CUDA kernels

The solutions generated for all these backends were checked by comparing the generated file trees and verifying equivalent normalised kernel source text (for C/C++/CUDA based kernels), dependency trees, and kernel IR signatures. The kernel IR signatures represent the translator’s internal intermediate representation, and consist of details like the kernel name, argument kinds, access modes, and named constants. For all example applications, the file trees, kernel source, dependency trees, and kernel IR were all verified to match between the Flang and `fparsr2` paths for every backend, providing evidence that both translation pipelines generated equivalent solutions on this suite. The generated Fortran source code did not match byte-for-byte, but this was expected as the Flang path slices the cooked kernel text while the `fparsr2` path pretty prints the AST. The MPI variants of each target backend also successfully matched these properties.

To further verify equivalence, each application was translated to the `c_cuda` backend using both the Flang and `fparsr2` pipelines and run via WSL. The solution runtimes, throughput (GFLOP/s) and bandwidth utilisation were all measured as within 2% of each other, as shown in Table 3. Each run was also compared on the results produced by the application, including `airfoil`’s RMS error against a reference solution, and the mass conservation sums and maxima reported by `mesh_res` and `tri_diff`. These results agreed to a relative difference of 0% between the Flang and `fparsr2` paths, providing a direct check that the generated kernels compute the same answers. These performance and application results back up the evidence of equivalence from the static analysis of the generated solutions.

Application	Runtime (s)		GFLOP/s		BW util (%)	
	<i>fparsr2</i>	<i>Flang</i>	<i>fparsr2</i>	<i>Flang</i>	<i>fparsr2</i>	<i>Flang</i>
<code>airfoil</code>	5.63	5.66	25.5	25.4	1.74	1.73
<code>mesh_res</code>	24.18	24.19	125.9	126.1	4.31	4.32
<code>tri_diff</code>	23.62	23.60	127.2	127.4	3.09	3.10

Table 3: Solution performance over the test suite

6.1.2 Validation Equivalence

The second stage of the project aimed to implement the validation step of the translation pipeline for the Flang path, with equivalent checks to the existing `fparsr2` validator - described in Section 5.2. To assess the success of this goal, a suite of 19 mini-apps were used, each constructed to exercise one of these checks; the checks being kernel resolution, unknown dependencies, parameter/const name conflicts, const writes, slice/stride incompatibility, `OP_READ` / `op_arg_idx` writes, `OP_INC` linearisation, and runtime-dimension local arrays, together with a clean kernel used as a control case.

Each case was translated with both the Flang and `fparsr2` pipelines to the sequential (`genseq`) backend. On the Flang path these cases carry `flang_body` on the kernel and its known dependencies, so they run through the native Flang validator walking the JSON expression trees, rather than lazily attaching an `fparsr2` AST and using the original walker. The driver classified each run as one of the four following outcomes, matching the behaviour described in Section 5.2:

- `OpError`: a missing or ambiguous kernel, or arity mismatch

- `loop.fallback = True`: translation exits successfully with `loop.fallback = True`, so no target-specific kernel is generated (sequential kernel is used instead)
- `warning`: translation exits successfully with a validation warning
- `pass`: a clean translation with no validation warning

The outcomes of the test suite are summarised in Table 4. The Flang and fparser2 pipelines produced equivalent validation outcomes on 18 of the 19 cases, including the same outcome class, warning or error kinds, and linearisation labels for the `OP_INC` cases.

Check	<i>fparser2</i>	<i>LLVM Flang</i>
Kernel resolution (missing, ambiguous, arity)	<code>OpError</code>	<code>OpError</code>
Read-only writes (<code>OP_READ</code> , <code>op_arg_idx</code> , child-subroutine)	<code>loop.fallback</code>	<code>loop.fallback</code>
Const writes	<code>loop.fallback</code>	<code>loop.fallback</code>
Slice/stride (dat and global)	<code>loop.fallback</code>	<code>loop.fallback</code>
<code>OP_INC</code> violations	<code>loop.fallback</code>	<code>loop.fallback</code>
Parameter/const name conflict	<code>warning</code>	<code>warning</code>
Runtime-dimension local arrays	<code>warning</code>	<code>warning</code>
Unknown dependencies	<code>loop.fallback</code>	<code>pass</code>
Clean kernel (control)	<code>pass</code>	<code>pass</code>
Well-formed <code>OP_INC</code> (control)	<code>pass</code>	<code>pass</code>

Table 4: Validation check equivalence between fparser2 and Flang

These tests confirmed that the kernel resolution validation check still raises an `OpError` with an identical message on the Flang path, and likewise the body walking checks correctly fall back to the sequential kernel when they flag an issue writing to an `OP2` const or a slice/stride incompatibility. Similarly, the two checks that are expected to emit a warning, a dummy parameter name that is also an `OP2` const and a locally declared array whose shape bounds reference a kernel parameter or `OP2` const, emitted the correct warning message and completed translation successfully. The control cases of a clean kernel and a well-formed `OP_INC` correctly passed on both paths.

The child-subroutine write check confirms that the Flang validator’s recursion through the `calls` entries in `flang_body` matches the fparser2 call-graph walk. The `OP_INC` labels also matched between the two paths, providing evidence that the name based disambiguation used in the Flang path validator is sufficient for these access-mode checks.

The only validation check that disagreed between the two paths was `unknown_dep`, a kernel that calls a subroutine not defined in the application. The fparser2 validator correctly set `loop.fallback` with an unknown dependencies warning, whereas the Flang path translated the same source as a clean pass, with no fallback or warning. This issue was caused by the Flang path initially filtering the raw `depends` names from the scanner against the set of known `Function` entities across the application, dropping any name that does not resolve. This means that by the time the validator runs, the unknown callee has already been removed, so the check condition doesn’t activate.

This was resolved by updating the filter to additionally retain names that appear as `CALL` in `flang_body["calls"]` even when they do not resolve - the behaviour described in Section 4.7.2. This allows `extractDependencies` to correctly report them as unknown and therefore fall back to the sequential kernel, consistent with the fparser2 path, while `FunctionReference` names that are only array indexing or unresolved intrinsics are still dropped to avoid falsely classifying these as missing callees. After this fix, the Flang path matches the fparser2 validator on all 19 cases.

6.2 Robustness

To assess the robustness of the Flang translation pipeline, compared to fparser2, the following testing categories were used:

- **Standards Coverage** - examples using syntax from Fortran standards 2008, 2018 and 2023 that may be outside the supported scope of fparser2
- **Pipeline Robustness** - testing the robustness of the translation pipeline on a variety of common and edge cases
- **Regression Tests** - searching for any issues introduced by the Flang implementation that force fallback to the fparser2 path

6.2.1 Standards Coverage

The following minimal cases test standards coverage of the Flang pipeline compared to fparser2, which officially claims support for Fortran 2003 plus some of Fortran 2008. While not representing a comprehensive conformance suite, these cases give a broad overview of the improvements in standards coverage provided by the Flang pipeline.

Fortran 2008

The following cases test Fortran 2008 features against both translation paths, with the results shown in Table 5.

Test Case	<i>fparser2</i>	<i>LLVM Flang</i>
coarray_decl	Fail	Pass
complex_re_im	Pass	Pass
contiguous_dummy	Pass	Pass
critical_construct	Pass	Pass
submodule_module_procedure	Fail	Pass
sync_all	Fail	Pass
sync_memory	Fail	Pass

Table 5: Fortran 2008 standards coverage cases

- **coarray_decl** - Tests the entity-decl coarray suffix (`real(8) :: buf[*]`) together with `sync all`. fparser2 accepts the `CODIMENSION [*]` attribute form, but not this suffix on the entity name.
- **complex_re_im** - Tests the F2008 complex %RE designator (`alpha = z%re`). fparser2 has no dedicated complex-part designator rule, but the ordinary % structure component production accepts this form, so both parsers complete translation.
- **contiguous_dummy** - Tests the F2008 CONTIGUOUS dummy attribute (`real(8), contiguous, intent(inout) :: x(:)`). Correctly handled by both paths.
- **critical_construct** - Tests the F2008 CRITICAL / END CRITICAL construct. fparser2 includes this in its F2008 grammar, so the guarded branch parses on both paths.
- **submodule_module_procedure** - Tests a parent module interface with a `submodule` containing a `module procedure` / `end procedure` body. fparser2 parses `SUBMODULE` and the `module subroutine` body form, but has no separate module subprogram production for this form. Flang does implement this form, so all program units parse successfully on the Flang path.

- **sync_all** - Tests the `sync all` statement (an image barrier). `fparser2` has no rule for this keyword sequence, whereas `Flang` includes `SYNC ALL` in its grammar, so the guarded branch only parses successfully on the `Flang` path.
- **sync_memory** - Tests the `sync memory` statement (a memory fence for atomic and coarray semantics). `fparser2` has no rule for this statement, but it works with `Flang`, which includes it in its grammar.

These cases confirm that the `fparser2` 0.2.4 path can correctly handle several Fortran 2008 features (like `CONTIGUOUS`, `%RE/%IM`, `CRITICAL`), but syntax gaps remain (the entity-decl coarray suffix, `SYNC ALL`/`SYNC MEMORY`, and the `MODULE PROCEDURE` body form). These gaps are all resolved by the `Flang` path, as `Flang` fully covers Fortran 2008, showing a significant improvement in standards coverage compared to `fparser2` even on standards that have claimed support.

Fortran 2018

The following cases test Fortran 2018 features, which are outside the officially supported scope of `fparser2`, against both translation paths.

Test Case	<i>fparser2</i>	<i>LLVM Flang</i>
<code>assumed_rank</code>	Fail	Pass
<code>assumed_type</code>	Fail	Pass
<code>co_sum</code>	Pass	Pass
<code>do_concurrent_default_none</code>	Fail	Pass
<code>do_concurrent_local</code>	Fail	Pass
<code>do_concurrent_local_init</code>	Fail	Pass
<code>do_concurrent_shared</code>	Fail	Pass
<code>error_stop_quiet</code>	Fail	Pass
<code>event_post_wait</code>	Fail	Pass
<code>fail_image</code>	Fail	Pass
<code>failed_images</code>	Pass	Pass
<code>form_team</code>	Fail	Pass
<code>image_status</code>	Pass	Pass
<code>implicit_none_external</code>	Fail	Pass
<code>import_all</code>	Fail	Pass
<code>import_none</code>	Fail	Pass
<code>import_only</code>	Fail	Pass
<code>select_rank</code>	Fail	Pass

Table 6: Fortran 2018 standards coverage cases

- **assumed_rank** - Tests Fortran 2018 assumed-rank arrays (`x(..)`), where a dummy argument may have any rank. `fparser2` rejects this because its grammar has no production for `(..)` in a type declaration. `Flang` supports assumed-rank arrays, so parsing succeeds and translation proceeds as expected.
- **assumed_type** - Tests Fortran 2018 assumed-type dummies (`type(*)`), which accept an argument of any type. `fparser2` does not recognise `type(*)` as a valid type-spec, whereas `Flang` accepts assumed-type dummies.
- **co_sum** - Tests `call co_sum(alpha)`, the F2018 collective intrinsic. This name is absent from `fparser2`'s intrinsic table, so the call parses as an ordinary subroutine reference on both paths and translates correctly.

- `do_concurrent_default_none` - Tests F2018 concurrent-locality clauses on `do concurrent ... default(none) shared(u)`. `fparser2` supports plain `DO CONCURRENT`, but not the locality list that follows the header, whereas Flang implements the full F2018 locality syntax. Therefore, the loop parses correctly on the Flang path, but fails on the `fparser2` path.
- `do_concurrent_local` - Tests the same locality extension, but with `local(t)` declaring a per-iteration local. `fparser2`'s `DO CONCURRENT` rule accepts only a header after `CONCURRENT`, so the leftover `local(t)` specifier causes the match to fail. Flang accepts `LOCAL` specifiers, so parsing succeeds.
- `do_concurrent_local_init` - Tests `do concurrent ... local_init(t)`, which copies the outer value of `t` into each iteration. `fparser2` has no `LOCAL_INIT` production, but Flang accepts the specifier.
- `do_concurrent_shared` - Tests the simplest F2018 locality specifier, `do concurrent ... shared(u)`. Even this form is outside `fparser2`'s `DO CONCURRENT` production, causing translation to fail on the `fparser2` path.
- `error_stop_quiet` - Tests `error stop 1, quiet=.true.`, where the F2018 `QUIET=` argument suppresses the stop message. `fparser2` only allows older `ERROR STOP` forms without that named argument. Flang recognises the F2018 form, so the guarded branch parses.
- `event_post_wait` - Tests F2018 event variables (`event post / event wait on an event_type`). `fparser2` has no parse rule for these statements; Flang's coarray-related F2018 support includes them, so the guarded branch is accepted.
- `fail_image` - Tests the F2018 `fail image` statement, which marks the current coarray image as failed. `fparser2` has no parse rule for this keyword sequence, whereas Flang's coarray-related F2018 support includes it, so the construct is accepted on the Flang path.
- `failed_images` - Tests `failed_images()`, the F2018 intrinsic that returns failed coarray images. Both paths accept the call as an ordinary function reference.
- `form_team` - Tests the F2018 team construct (`form team, change team, end team`). `fparser2` has no parse rules for these statements; Flang parses the full sequence as valid F2018 syntax.
- `image_status` - Tests `image_status(1)`. As with the other F2018 intrinsic names, both paths accept it.
- `implicit_none_external` - Tests `implicit none (type, external)`, the F2018 parenthesised form that disables implicit typing for types and externals separately. `fparser2` only accepts plain `IMPLICIT NONE`; Flang implements the F2018 variant and parses successfully.
- `import_all` - Tests `import, all`, a F2018 form that imports every host association into an internal procedure. `fparser2` only implements the Fortran 2003 `IMPORT / IMPORT :: name-list` forms, so the F2018 comma-selector syntax is rejected, whereas Flang accepts the F2018 import forms.
- `import_none` - Tests `import, none`, the strictest F2018 `IMPORT` form. `fparser2` rejects it for the same reason; the F2003 import rule has no comma-selector production.
- `import_only` - Tests `import, only: a`, selectively importing a host name. `fparser2` again rejects the F2018 comma-selector form, whereas Flang's F2018 import support includes selective `ONLY` imports.

- `select_rank` - Tests assumed-rank `x(..)` together with `select rank / rank (1) / rank default`. Both features are F2018 and absent from `fparser2`'s grammar. Flang implements the full rank-selection construct, so parsing succeeds.

These cases confirm that Fortran 2018 syntax sits almost entirely outside `fparser2`'s supported scope. The only cases that pass on both paths are the three F2018 intrinsic names (`co_sum`, `failed_images`, and `image_status`), which both pipelines accept as ordinary procedure references. All remaining constructs fail to parse under `fparser2`, but pass on the Flang path, again showing a significant improvement in standards coverage provided by the Flang pipeline.

Fortran 2023

Similar to the previous set of cases, these test features that sit outside the supported scope of `fparser2` from Fortran 2023.

Test Case	<i>fparser2</i>	<i>LLVM Flang</i>
<code>conditional_expr</code>	Fail	Pass
<code>do_concurrent_reduce</code>	Fail	Pass
<code>notify_wait</code>	Fail	Pass

Table 7: Fortran 2023 standards coverage cases

- `conditional_expr` - Tests a Fortran 2023 conditional expression (`flag > 0 ? 2.0d0 : 1.0d0`), analogous to a ternary operator. `fparser2`'s grammar has no rule for this token sequence, whereas Flang accepts conditional expressions, so parsing succeeds.
- `do_concurrent_reduce` - Tests `do concurrent ... reduce(+:s)`, the Fortran 2023 reduction locality form. `fparser2` has no grammar for `reduce(+:...)` after the concurrent header, whereas Flang implements `REDUCE` locality and correctly parses the construct.
- `notify_wait` - Tests the Fortran 2023 `notify wait` statement on a `notify_type`. `fparser2` has no production for this statement; Flang accepts it, so the guarded branch parses.

Across all the standards cases tested, a majority of cases fail under `fparser2`, while every case passes on the Flang path. This represents a marked improvement in the robustness of the translation pipeline.

6.2.2 Pipeline Robustness

Pipeline robustness cases were developed to check the ability of the Flang pipeline to handle a variety of both common and edge case OP2 applications developed in Fortran (without needing to fall back to `fparser2`), compared to the existing translation pipeline. The tests covered OP2 application components like mapped `op_arg_dat` with `OP_INC`, global reductions, `op_arg_idx` and `op_arg_info`, optional arguments, `:soa` type strings and `r8` aliases, mixed-case API names, `op_decl_const` from a used module, and multiple `op_par_loop` sites in one file.

On almost all of these cases both backends pass, with Flang passing on all cases where `fparser2` passes whilst remaining native. This is strong evidence that the Flang pipeline provides equivalent robustness to `fparser2`; it does not drop loops, misclassify access modes, or generally require fallback to `fparser2`.

Two related edge cases were found where the existing `fparser2` pipeline fails: `pointer_init_target` and `array_pointer_init`. In these cases, the application initialises a Fortran 2003 data pointer to a named target, in the respective forms:

```

pointer_init_target:  integer, pointer :: ptr => var
array_pointer_init:  integer, pointer :: p(:) => a

```

This is valid under the Fortran 2003 standard fparser2 claims to support, but it nevertheless rejects both declarations at parse time, so translation fails. A control form of this syntax, `integer, pointer :: ptr => null()`, does correctly parse with fparser2 - so the failure was identified as a parser bug on named target initialisation. Conversely, LLVM Flang accepts both forms, so both cases are successfully translated on the Flang path.

This issue exemplifies the stronger robustness of the Flang pipeline over fparser2 outside of simple gaps in standards coverage, showing its ability to robustly handle edge cases where fparser2 fails, even within its claimed standard. In addition, as a full compiler-grade parser with a full standards implementation and better maintenance, these improvements in robustness should effectively generalise to resolving any other existing or future issues that could affect the fparser2 pipeline.

6.2.3 Regressions and Fixes

To check that the Flang path works correctly and equivalently to the fparser2 path in cases that work with the existing translation pipeline, and validate that the Flang path doesn't fall back to fparser2 unnecessarily, a set of regression tests were created in the form of valid OP2 mini-apps. These covered key areas, such as C-preprocessor macros and `#include` of loop sites, vector `OP_INC` patterns, and validation warnings that should still exit successfully with `loop.fallback`. In addition, an intentionally broken stub of the scanner was used to create scenarios where fallback to fparser2 was required, and verify this process worked correctly.

Most of these cases passed correctly when using both paths, but there was one notable regression case observed on the Flang path, which worked correctly when using the fparser2 path. This case was using Fortran `INCLUDE`; unlike C-preprocessor `#include`, which is expanded before the scan, `INCLUDE "kernel_site.inc"` is left in the free-form source code and therefore needs to be resolved by the Fortran parser. On the Flang path, the Python driver preprocesses the source file and invokes the scanner with that text on stdin. Because Flang's parser API needs to read source code from a real file, rather than an in-memory string, the scanner is designed to write the stdin body to a temporary file under `/tmp`. When the Flang prescan hit `INCLUDE "kernel_site.inc"`, it searched relative to that temporary directory rather than the real application directory containing the `.inc` files, therefore reporting that the include could not be found, and causing a fallback to fparser2. This fallback allowed the translation to still complete successfully, as its `FortranStringReader` is given the translator's `include_dirs`, so it can open `.inc` files even when the input is an in-memory string. Despite the fallback preventing an absolute translation failure, this issue nevertheless represents a problem with the Flang pipeline that should not occur.

This regression was fixed in `op2-flang-scan` by writing the temporary file next to the original source directory, so Flang's file directory lookup finds the corresponding `.inc` files. Additionally, the directory is added to Flang's `Options.searchDirectories`, so it can be used for `-I` resolution, matching the include set already passed to fparser2. On the Python side, `run_scan` forwards the `include_dirs` using the `-I` flag and sets the subprocess working directory to the source file's parent as an additional safeguard. After this change, the cases using Fortran `INCLUDE` all pass natively without fallback.

6.3 Performance

6.3.1 Translation Performance Breakdown

The performance and overhead associated with the Flang and fparser2 translation paths were evaluated by measuring the breakdown of run times of different stages of the translation process for each pipeline, shown in Table 8. Specifically, the runtime for the Flang translation pipeline is broken into the complete runtime (Translation), the time Flang spent parsing the source code (Parse), the time spent walking the parse tree and building the JSON output (Parse Tree Walk), the time to deserialise and load the JSON on the Python side (JSON Load), and the overhead associated with subprocess spawning and inter-process communication (Spawn/IPC). Materialisation refers to the scanner writing the preprocessed source to a temporary file, for use with Flang’s parser API.

The performance breakdown was performed across five mini-applications, with `airfoil` and `scale_mesh` being multi-file applications, and `mesh_res`, `scale_mesh_flat`, and `tri_diff` being single-file applications. `scale_mesh` is a deliberately larger application, with its translation units being split across ten files, and `scale_mesh_flat` is an identical application but with these translation units all combined into a single file, to help assess scalability.

Example	LLVM Flang							fparser2	
	Translation (ms)	Parse (ms)	Parse Tree Walk (ms)	JSON Load (ms)	Spawn/IPC (ms)	Materialise (ms)	Parse Time % of Translation	Translation (ms)	Parse (ms)
<code>airfoil</code>	394.16	230.93	18.41	1.17	96.11	5.12	58.6%	193.43	189.71
<code>mesh_res</code>	143.57	85.36	6.80	0.31	34.17	1.77	59.5%	94.77	93.26
<code>tri_diff</code>	130.55	78.44	6.96	0.27	30.10	1.66	60.1%	77.46	76.21
<code>scale_mesh</code>	1477.83	874.32	90.11	25.07	316.76	17.87	59.2%	807.96	778.01
<code>scale_mesh_flat</code>	234.86	150.03	29.25	23.07	14.48	2.14	63.9%	746.71	719.50

Table 8: Translation performance breakdown for the Flang path

Note: values are means of five timed runs

From these results, it can be observed that for standard multi-file applications, like `airfoil` and `scale_mesh`, the runtime of the translation process is significantly higher with Flang than fparser2. This is primarily because the scanner is spun up separately for each individual file, which multiplies the already significant overhead associated with the Flang path. Each invocation adds time for subprocess spawning, materialising a temporary file, and loading the LLVM binary, which scale linearly with the number of files. For example, Spawn/IPC is around 14–34 ms on a single-file application, compared to 96 ms on `airfoil`, and 317 ms on the ten-file `scale_mesh`. This issue is also clearly evidenced by comparing the breakdowns of `scale_mesh` and `scale_mesh_flat`. The multi-file version of the application takes almost twice as long to complete translation as fparser2 (1478 ms vs 808 ms), but the single-file `scale_mesh_flat` runs significantly faster using the Flang pipeline (235 ms vs 747 ms), showing the issue of high overhead associated with the Flang pipeline on multi-file applications. The performance of fparser2 is similar across both versions (808 ms and 747 ms), and significantly slower than Flang on `scale_mesh_flat`, which shows the performance limitations of fparser2 when dealing with larger application sizes.

For all example applications, the proportion of translation time spent parsing on the Flang path is relatively consistent at around 60%, showing the parse time also worsened somewhat in proportion with the overhead as the number of translation units increased. This is also shown by `scale_mesh` taking 874 ms to parse compared with 150 ms for `scale_mesh_flat`, and the parse-tree walk following the same pattern (90 ms vs 29 ms). This is likely to simply be an artifact of the repeated Flang frontend setup in each scanner process, which was counted as part of the parse time, rather than the parsing itself showing degradation. Separately, the results show that JSON loading scales with the application size rather than file count (25 ms vs 23 ms

on the two `scale_mesh` variants), so it is not a scalability concern for multi-file applications.

For the smaller single-file applications `mesh_res` and `tri_diff`, the parse time is roughly equivalent between the Flang and `fparser2` paths. The remaining performance gap is the overhead of the Flang path, including Spawn/IPC of around 30 ms, so the performance of these applications remains modestly worse than that of the `fparser2` implementation.

6.3.2 Introducing Batching

To mitigate the performance issues on applications with multiple files, a concept of batching was added to the Flang translation pipeline. Previously, the Flang pipeline spawned separate `op2-flang-scan` subprocesses for every source file, so applications with multiple translation units such as `airfoil` and `scale_mesh` induced additional overhead related to subprocess spawning, LLVM binary loading, and Flang frontend setup. With the addition of batching, the Python driver preprocesses every file in the application, then invokes a single scanner subprocess that takes key information (such as the path, byte count, and source body) for each translation unit via `stdin`. Inside the C++ process, the scanner still runs `Prescan` $\rightarrow$ `Parse` $\rightarrow$ `walk` $\rightarrow$ `JSON emit` sequentially for each unit, as Flang cannot parse multiple translation units in a single call. However, this means only a single subprocess is spawned, and consequently there is only one load of the LLVM binary and setup of the Flang frontend, which drastically reduces the pipeline overhead. Once run, each translation unit’s JSON object is written to `stdout` on its own line, which Python deserialises to build the `Program` objects, and per-file Flang failures still produce a JSON `"error"` field so the driver can fall back to `fparser2` for that file only. Batching is optional via a flag on the scanner call, but is enabled by default for the Flang pipeline.

Example	LLVM Flang								fparser2	
	Translation (ms)	Parse (ms)	Parse Tree (ms)	Walk (ms)	JSON Load (ms)	Spawn/IPC (ms)	Materialise (ms)	Parse Time % of Translation	Translation (ms)	Parse (ms)
<code>airfoil</code>	181.04	105.14	8.71		1.11	32.38	12.57	58.1%	193.43	189.71
<code>mesh_res</code>	132.65	80.10	6.27		0.27	27.53	3.71	60.4%	94.77	93.26
<code>tri_diff</code>	143.72	85.00	7.07		0.26	31.59	4.10	59.1%	77.46	76.21
<code>scale_mesh</code>	381.60	234.47	29.07		19.90	13.82	42.11	61.4%	807.96	778.01
<code>scale_mesh_flat</code>	236.62	150.91	29.10		21.70	12.07	4.52	63.8%	746.71	719.50

Table 9: Translation performance breakdown with batching

Note: values are means of five timed runs

The performance breakdown with batching enabled is shown in Table 9. By amortising the pipeline overhead, the improved performance of Flang that was anticipated in Section 2.2.2 starts to be observed in practice, as, for multi-file applications, the batching approach significantly improves the performance of the Flang pipeline. It reduces the times for Spawn/IPC (96.11 ms $\rightarrow$ 32.38 ms for `airfoil`) and the parse tree walk (18.41 ms $\rightarrow$ 8.71 ms), as well as the overall parse time (230.93 ms $\rightarrow$ 105.14 ms), with the JSON loading remaining relatively constant as expected. This means the Flang path now performs better than `fparser2` for multi-file applications, including both `scale_mesh` (translation time of 381 ms vs 808 ms) and `airfoil` (181 ms vs 193 ms), with the trend being that the larger the application, the more significant the performance improvement. On smaller applications like `mesh_res` and `tri_diff`, however, the performance of the Flang path remains worse, due to the overhead playing a more significant role. For example, the translation time of `mesh_res` was 132 ms, compared to 94 ms with `fparser2`. Nevertheless, this trend of ‘better performance relative to `fparser2` the larger an application is’ represents a significant improvement to the translation pipeline for real-world applications. The relative performance comparisons between the Flang and `fparser2` paths (with and without batching) across all example applications are visualised in Table 10.

Example	Files	Translation time (ms)			Latency vs fparser2	
		Standard	Batched	Latency w/ batching	Standard	Batched
airfoil	3	394.2	181.0	0.46×	2.04×	0.94×
mesh_res	1	143.6	132.7	0.92×	1.51×	1.40×
scale_mesh	10	1477.8	381.6	0.26×	1.83×	0.47×
scale_mesh_flat	1	234.9	236.6	1.01×	0.31×	0.32×
tri.diff	1	130.6	143.7	1.10×	1.69×	1.86×

Table 10: Batching scalability

The core limitation of batching is that the Flang parser can only parse one translation unit at a time, so the batching process is not a true multi-file parse and only amortises the overhead of the Flang path, not improving the parsing efficiency itself. The only significant downside of batching is related to failure isolation in specific scenarios. If the translation process results in a segmentation fault when using batching, the entire batch will fail, whereas with per-file spawning (non-batching) only a single file will fail, which can be more easily debugged.

6.4 Practicality

The practicality of using the Flang path primarily comes down to the cost of installing the toolchain. Unlike fparser2, which is a PyPI package that is lightweight and easily installable via pip, LLVM Flang requires a matching LLVM ≥ 23 install that exposes the Flang parser headers and libraries (see Section 4.7.3). In addition, the full dependency tree can require on the order of 20–30 GB of disk space, and needs to be installed using a from-source build (or via a tool like `apt-get` or `brew`, if a compatible version is available) which can be a time-consuming process and introduces complexity around package names and CMake prefixes which can vary across LLVM versions.

After this setup process, use of Flang within OP2 is relatively straightforward. The `--parser` flag has been introduced to the existing Python translator to allow simple switching between the Flang and fparser2 pipelines (this can also be set via an environment variable). Invoking with `--parser flang` locates `op2-flang-scan` (via a flag, an environment variable, a default build path, or `PATH`) and runs the scanner as a subprocess. Due to the more complex setup process for Flang, fparser2 remains the default parser, which is still a viable choice for the majority of users and avoids any additional setup work. Likewise, a failed Flang scan falls back to fparser2 for that file, which avoids any possible regressions introduced by adopting the Flang path.

While the setup can be a tedious process, the benefits of the Flang path accrue for the robustness improvements on every translation that fparser2 cannot handle, and on larger multi-file applications where the Flang path can lead to significant performance improvements in the translation process. The users for whom this trade-off is attractive are likely to be those already operating in a HPC environment, such as research groups running large unstructured-mesh simulations on clusters. For this audience a site-wide Flang module is a familiar kind of dependency, so this may not be considered a significant burden, and the benefits introduced by the Flang path would be a notable improvement over the existing fparser2 path. In contrast, more casual users working with smaller OP2 applications would likely not benefit as much from the performance improvements for larger applications, and are less likely to need the robustness of the Flang path, so may be better served by the existing (default) fparser2 path.

7 Codebase Integration

7.1 Implementation Refinement

Following the evaluation of the Flang pipeline, the implementation was refined to production grade so it was ready for a pull request into the live OP2 codebase. The key efficiency improvement introduced was batching, discussed in Section 6.3.2.

To improve the readability and maintainability of the C++ scanner, the code was annotated with comments and Doxygen-style docstrings [35] on public types and functions, covering the JSON contract, the parse-tree walk, and the batch protocol. Doxygen was chosen as the standard due to its widespread adoption as a formatting specification for source code documentation. The ClangFormat tool [36] was used to format the code based on the LLVM style, including brace and indent rules.

Error handling was also expanded to give transparency into issues that occur during the translation process. The scanner emits a JSON object with an `"error"` field for any unit that fails to parse, so the Python driver can fall back to `fparser2` for that file, and the translator prints a warning on each such fallback. Fatal cases, like a missing scanner binary, invalid JSON, or syntax that `fparser2` also cannot handle, still surface as hard errors which are clearly explained to the user.

Finally, the documentation was updated to reflect the new Flang path along with its setup, build process, and the usage of the updated translator.

7.2 Setup and Build

The Flang translation pipeline, implemented as a C++ scanner, is incorporated into OP2 as an optional step of the existing library setup, with the ‘Getting Started’ documentation [34] updated to list CMake ≥ 3.20 and LLVM Flang ≥ 23 alongside the existing optional build dependencies (like MPI, CUDA, HIP, HDF5, and the mesh partitioners). It also adds a new step to export the `LLVM_INSTALL_PATH` in the same path export step already used for those libraries. Users who do not intend to use the Flang path can continue without these packages, and the OP2 libraries still build as before.

As described in Section 4.7, using the tuple-trait layout of Flang’s parse-tree API imposes a version floor, so the generalised requirement is LLVM Flang ≥ 23 , together with the Flang parser headers, the parser libraries (`libFortranParser` and the related `Fortran*` archives), the LLVM CMake package (`lib/cmake/llvm/LLVMConfig.cmake`), and CMake ≥ 3.20 . These are all detected during the standard `make -C op2 config` pass. A full Flang compiler driver is not required, but when Flang itself is built from source MLIR must still be enabled to successfully complete the build.

`common.mk` includes the added `flang.mk` and reports LLVM Flang in the same configure log as HDF5 and the partitioners. `flang.mk` looks for `cmake` on `PATH` and an LLVM prefix that contains the parser headers and LLVM CMake package. The prefix search starts from the `LLVM_INSTALL_PATH`, then likely locations, including `$HOME/.local/llvm`, the prefix reported by `llvm-config`, versioned paths, and a `/usr` install. When both CMake and a valid prefix are visible, the configure log records `HAVE_FLANG` and the detected prefix, printing `LLVM Flang FOUND`; otherwise reporting that Flang was not found (or that CMake was missing) and the library build continues without the scanner.

The scanner’s CMake keeps the link surface to the parser libraries to avoid linking the full Flang compiler driver, which is not required as the scanner only performs prescan and parse, and does not run semantic analysis, MLIR lowering, or code generation. It does not call `find_package(Flang)`, as the packaged `FlangConfig.cmake` unconditionally imports Clang

targets even when only the parser is needed, which would fail when LLVM was built without Clang in `LLVM_ENABLE_PROJECTS`. Instead the CMake file finds LLVM for header paths and `LLVMSupport`, then links the Flang parser libraries directly. The parser also depends on LLVM's frontend directive helpers (`LLVMFrontendOpenMP`, `LLVMFrontendOpenACC`, and related libraries), which are not tracked by Flang's own link interface and are therefore added manually when they exist.

Obtaining a matching install prefix is the main additional setup step, and is documented in the same 'Getting Started' documentation [34]. On Debian and Ubuntu the distribution archives currently stop at LLVM 20, which is not compatible with the scanner (Section 4.7.3). An appropriate version can instead be installed, for example, via the LLVM project's APT repository [37], Fedora or Homebrew - or built from source.

Once the prefix exists, the same `make -C op2` command as before can be used to build the OP2 libraries. When `HAVE_FLANG` is set, the library Makefile adds a `flang-scan` prerequisite to the default `all` target, and configures the scanner with CMake against the detected prefix. This builds the scanner under the OP2 object directory and installs the binary next to the libraries at `op2/bin/op2-flang-scan`. Users who only need the helper can run `make -C op2 flang-scan`; if Flang was not found, an error is printed referring to the `LLVM_INSTALL_PATH` and documentation.

Fortran applications can then be translated with the Flang path by setting `OP2_FORTTRAN_PARSER` to `flang` before the usual application Make target. `f_app.mk` forwards that as `--parser flang` and points `OP2_FLANG_SCAN` at the built binary. This can also be passed as extra translator flags or on a direct translator invocation. Because the default translation pipeline remains `fparser2`, users who don't install Flang see no change in behaviour.

8 Project Review

8.1 Development and Management Methodologies

The project was delivered in an asynchronous manner, with the goals and motivation for the project initially discussed and laid out with the supervisor, while the timelines and more fine-grained goals were handled independently. Supervisor check-ins were done for key inflection points of the project, namely handling the issue of LLVM Flang being such a heavy dependency and how this should be handled with regard to the direction of the project from that point.

The development methodology used throughout the implementation was a form of extreme programming, where new concepts and ideas were rapidly prototyped and evaluated to assess their viability, before being refined into a complete solution. This assisted with maintaining rapid progress through the project and making key design decisions. The evaluation used a more measured approach, where the test suite was first crafted against a variety of Fortran standards and mini-applications, before running them against the Flang pipeline. Throughout the development, the code changes were managed through Git, with branches being used for prototyping feature development and refining into pull requests to merge changes together.

8.2 Outcomes and Timeline

The goal of the project, laid out following the literature review, was to integrate LLVM Flang into OP2 as an alternative Fortran parser to `fparser2`, and as such perform the critical functionality of this Flang translation pipeline solely using the information collected via Flang's parse-tree API. This involved the three stages laid out in Section 3.2, which were all successfully completed, including the third stage research extension.

Figure 8 shows the Gantt chart of the original project timeline as it was laid out from the outset in the research proposal. Upon starting the development for the project, the methodological approach was changed to working through the implementations of each stage before evaluating the pipeline as a whole, rather than separately implementing and evaluating each stage individually. This was done because the implementations of each stage are tightly coupled, so were difficult to discretely separate in a logical manner. Additionally, the evaluation of each stage would also involve significant overlap, so following a more linear development methodology of *Design* → *Prototyping* → *Evaluating* → *Refining* fit the progression of the project more naturally. The core stages were implemented ahead of schedule, including the research extension, with the evaluation also being successfully completed on time. In addition, the literature review, interim report and presentation were completed on schedule according to the anticipated timeline.

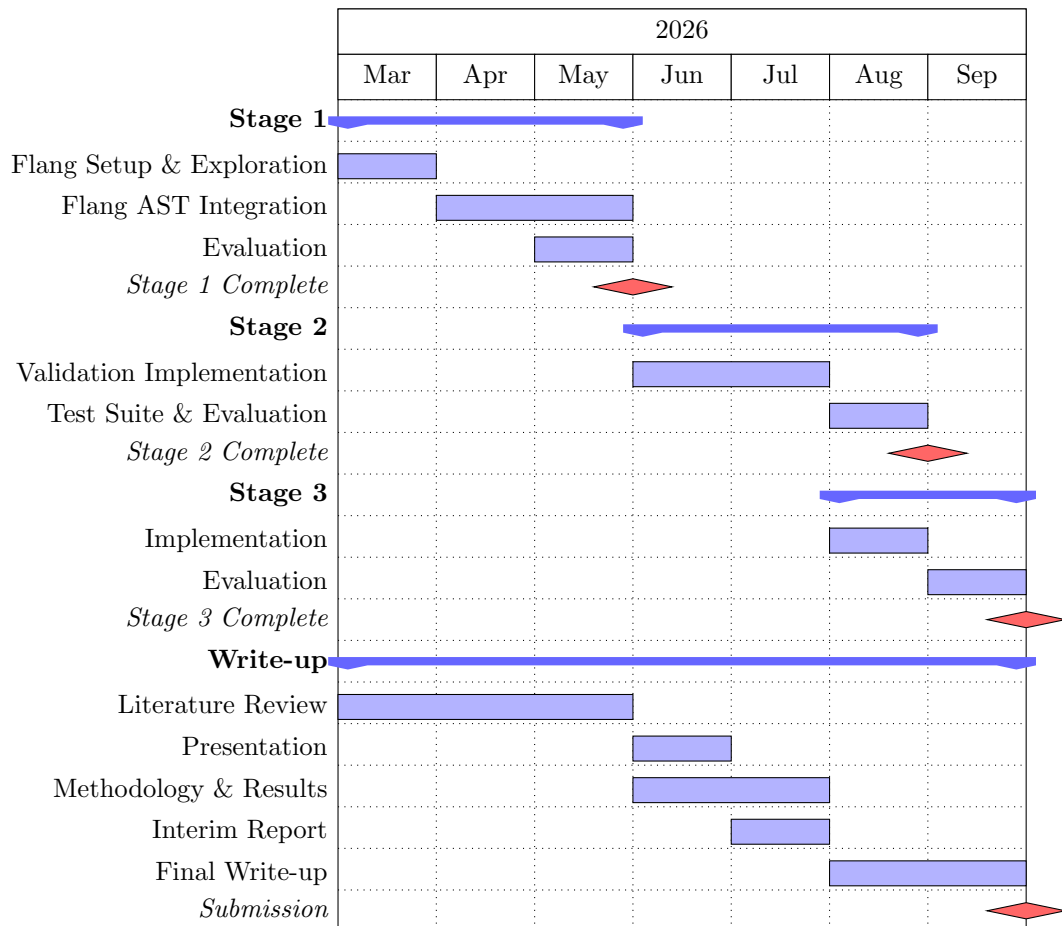


Figure 8: Original project timeline Gantt chart (from the research proposal)

Figure 9 shows the Gantt chart of the revised project timeline of the remaining work, projected from part of the way through the project in the interim report. At this point, the core stages of the project (1 through 3) had been completed in an initial form, with the core remaining work expected to be in evaluation, refinement and exploring alternative approaches. The evaluation was extensive and identified several issues, most notably performance issues with multi-file applications. The implementation refinement stages therefore involved a variety of fixes and improvements, including the introduction of batching to the scanner, on top of the improvements in maintainability, clarity and documentation. These steps took longer than expected, so there was not enough remaining time in the project to explore alternative approaches, so this was left as a key area for future work.

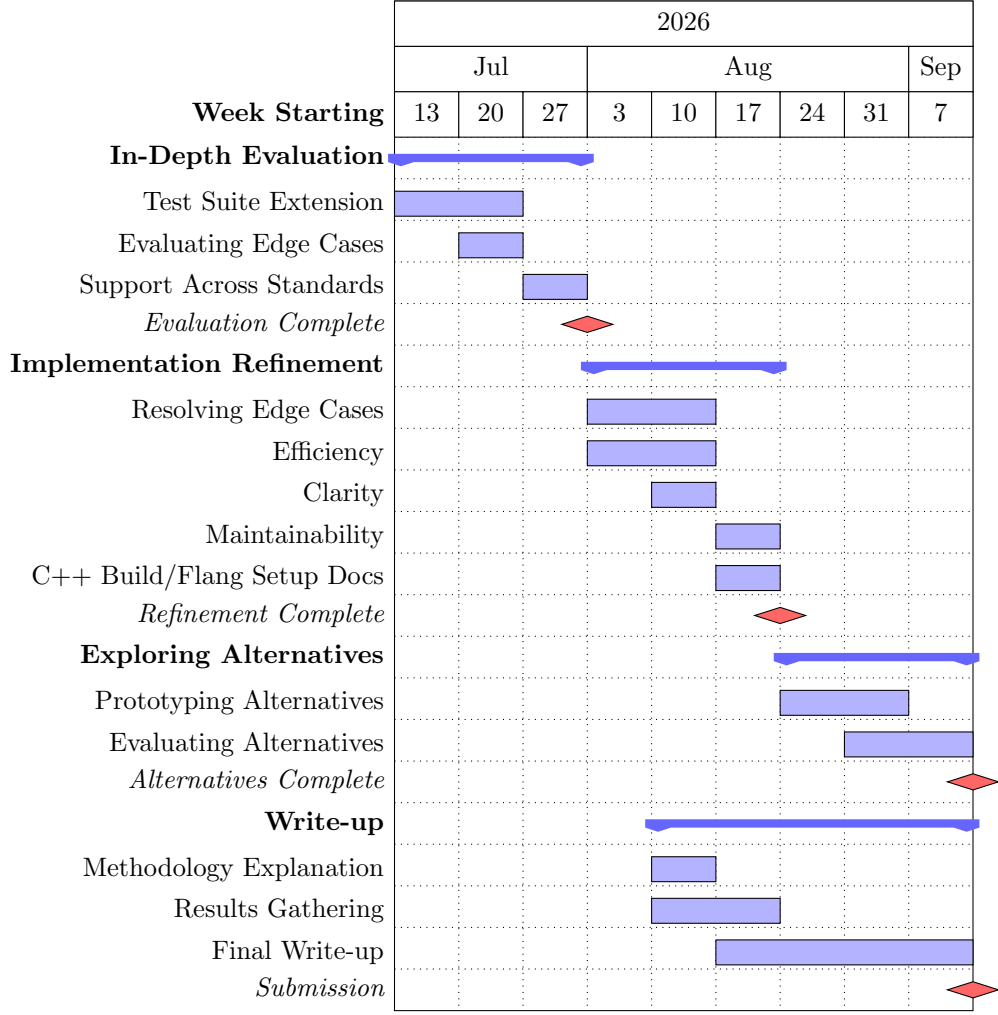


Figure 9: Revised project timeline Gantt chart (from the interim report)

8.3 Expected vs Observed Trade-offs

The literature review identified several potential benefits of using LLVM Flang over fparser2, relating to the robustness, performance, and maintainability of the translation pipeline, alongside acknowledged costs in practicality and integration difficulty (Table 1). The evaluation allows these hypotheses to be answered, based on evidence gathered, as either holding true or not materialising as expected.

The primary expected benefit was improvements in robustness. As a full compiler frontend supporting a wide variety of Fortran standards, Flang was predicted to be significantly more reliable and robust than fparser2, and provide better coverage of standards and language constructs. This proved to be the strongest demonstrated result, as most of the standards coverage cases in the robustness suite failed under fparser2 but passed on the Flang path (Tables 5–7). Broader robustness gains were also demonstrated, with the Flang path resolving issues within the standards that fparser2 claims to be able to handle, such as named target pointer initialisation within valid Fortran 2003 code. On standard OP2 pipeline cases, Flang matched the capabilities of fparser2 on extraction, access modes, and validation warnings; and remained native rather than requiring fallback to fparser2. Robustness was therefore shown to be significantly improved without introducing any regressions, matching the expected benefits laid out at the outset.

Performance was also expected to be a clear advantage, due to Flang being an efficient C++-

based compiler frontend rather than using interpreted Python. However, without batching, multi-file applications like `airfoil` and `scale_mesh` were significantly slower than `fparser2`, due to the per-file overhead of the scanner subprocess. Single-file applications had similar parse times, but the Flang translation was still modestly worse, again due to this overhead representing a significant portion of the total translation time. With batching introduced (Section 6.3.2), the Flang pipeline was able to scale more efficiently than `fparser2` to larger applications, for example, `scale_mesh` became substantially faster than `fparser2`, though smaller examples remained slower (Table 9). While this is not as strong a benefit as expected, it still represents a useful real-world benefit for larger applications, which naturally take longer to translate.

Practicality, in terms of dependency size and installation complexity, was anticipated as a major downside of using Flang, and this proved to be accurate. The scanner requires LLVM Flang ≥ 23 , which takes a significant amount of disk space and can be complex to install. Similarly, maintainability was rated only *Medium* in the literature review (Table 1), due to the anticipated complexity of the integration. In practice, maintainability appears likely to be even weaker than expected, as Flang’s parse-tree API is not stable, meaning breaking changes are likely to crop up in the future and will need to be addressed.

8.4 Unforeseen Issues

Several issues that were not anticipated during the initial research phases surfaced over the course of the project, and had to be dealt with effectively.

The first was the heaviness of the Flang toolchain, which became clear during the initial setup (Section 6.4). As this cost is inevitable in order to use the Flang parser within OP2, it was accepted while leaving Flang as an optional path and `fparser2` as the default, and linking only the parser libraries rather than a full compiler driver. This issue does significantly affect the practicality of the Flang pipeline, and therefore makes the possible alternatives highlighted in Section 2.2.3 of greater interest for future work.

A second issue was identified while searching for regressions in the robustness of the Flang pipeline, where Fortran `INCLUDE` of `.inc` files was failing Flang translation and causing fallback to `fparser2` (Section 6.2.3). This case shows the value of the fallback mechanism, as it allowed the translation to successfully complete despite the Flang path failing. While the evaluation was extensive, it was not exhaustive, so this fallback could be critical for maintaining the robustness of the OP2 translators if similar issues slipped through the cracks. On any such cases, a warning will also be issued to help identify and fix the issue in the Flang pipeline.

The performance evaluation then showed that multi-file applications such as `airfoil` and `scale_mesh` were slower than `fparser2`, contrary to the anticipated speedups, as each file in an OP2 application initiated a separate run of the scanner. Batching was formulated as a solution to this, which amortised the overhead of multiple scanner runs, allowing larger applications to become faster than `fparser2`, though small single-file examples remained slower, which was accepted as an inherent cost of the C++ interface architecture used for the Flang pipeline.

9 Conclusion

This research project addressed the gap in the literature for integrating Flang into OP2 in order to make the Fortran source-to-source translators more robust. All three stages of the project were successfully completed, with Flang integrated as an optional alternative translation path to `fparser2`. The material contributions of this work are:

- A C++ scanner, `op2-flang-scan`, that walks Flang’s parse tree and emits a JSON contract from which the Python translators can generate the targeted application code

- A Flang-based path through kernel validation and Fortran-to-C++ kernel translation, with lazy fallback to `fparser2` when a file or dependency cannot be handled to ensure no regressions are introduced over the existing pipeline
- An evaluation of equivalence, robustness, translation performance, and practicality of the Flang path against the `fparser2` pipeline
- Updates to the build and setup process, documentation, and user-facing interface reflecting the new Flang path
- A pull request of the full Flang integration into the live OP2 codebase

Reflecting on the research questions posed in Section 3.1, the evaluation showed the new Flang pipeline to be equivalent to the existing `fparser2`-based pipeline in terms of solution generation and validation checks performed, while significantly improving the robustness. Every standards coverage case passed on the Flang path, including constructs from Fortran 2008, 2018 and 2023 that `fparser2` failed to parse, and further pipeline robustness cases also proved Flang’s ability to handle edge cases where `fparser2` fails. Performance results were more mixed than expected, with larger multi-file applications such as `scale_mesh` translating substantially faster than `fparser2` (when using batching), while smaller single-file applications became modestly slower. There were, however, significant costs associated with the Flang integration - primarily in terms of practicality and maintainability. Flang is a very heavy dependency, requiring LLVM ≥ 23 and a large amount of disk space; and the complexity of the integration, along with the instability of Flang’s parse-tree API, means that maintainability is likely to be weaker than the `fparser2` pipeline. Due to the limited practicality, Flang was left as an optional pipeline choice, with switching available via flags or environment variables once the dependencies have been installed, which minimises the setup overhead required to get started with OP2.

Addressing the final research question, ‘How do the benefits and trade-offs of a Flang-based translation path weigh up for real users?’, the heavy dependency and tedious installation make it impractical for many users, but the benefits to robustness and performance on larger applications are clear. For independent users working with smaller OP2 applications, these benefits are likely not worth the cost. However, for more advanced users, or those working in a HPC environment, a large dependency is likely not a significant burden, and the robustness and performance gains should represent a noticeable improvement for their workflow over the existing `fparser2` path.

Beyond improving the OP2 eDSL, this research shows that a compiler frontend parser can deliver substantial robustness gains over Python-based parsing for HPC source-to-source translation, even without semantic analysis. It also shows the potential pitfalls of such an approach, such as an increased maintenance burden, complexity of the translation pipeline, and installation practicality issues. These represent useful insights for other DSLs, which are likely to face similar trade-offs when considering the integration of compiler frontends or other methods of improving translation robustness.

The final deliverable of the project was made in the form of a pull request to the main branch of OP2 encapsulating the full LLVM Flang integration [38].

9.1 Future Work

The outcomes of this research project leave two natural directions for future work. The first is exploring alternative parsers to Flang and `fparser2`, such as those considered in Section 2.2.3, and evaluating them based on their trade-offs. The second is extending the existing scanner with Flang’s semantic analysis, to further improve the robustness of the OP2 translators.

With the observed costs of the Flang integration, namely the heavy dependency and weaker maintainability than `fparser2`, alternative approaches have clear areas where they could outperform the Flang pipeline. In addition, as the Flang pipeline did not leverage semantic analysis to achieve the robustness improvements, it is likely that alternative parsers which do not include semantic analysis could still potentially achieve similar robustness improvements.

Tree-sitter is a particularly plausible candidate - it's a lightweight C parser with Python bindings that install via pip, meaning it could be substantially stronger than Flang in terms of practicality and integration difficulty, while having the potential to bring comparable performance. Some key open questions are whether the `tree-sitter-fortran` grammar can match Flang's Fortran 2018 and 2023 coverage closely enough to achieve similar robustness improvements, and whether the API is stable over time to improve the maintainability of the translators. Another possibility is LFortran, which is an interactive Fortran compiler with native Python bindings, which should make it easier to integrate than Flang, and a conda-based install which would make it more practical. At the time of the literature review, it was judged too unstable for production use, as it remained in alpha and was 'expected to not work on third-party code' [25]. This could be revisited once the compiler can parse third-party codes reliably, exposes a stable Python API, and covers Fortran 2018 closely enough to match the robustness gains observed here.

Separately, running Flang's semantic analysis, so that a symbol table is materialised before the parse tree is walked, remains a possible extension of the existing scanner. This would disambiguate array references from function calls, and derived-type constructors from calls inside the scanner; give more precise `depends` lists; and resolve `USE` association, `IMPLICIT` typing, and name collisions on the Python entity set. These gains would be valuable, but remain relatively narrow in scope compared to the robustness improvements already obtained from parse-only Flang, and a stricter semantic pass could increase the failure rate of Flang parsing (fallback to `fparser2`) if translation units are not fully typed. Investigating the value of this trade-off is left as a key area of exploration for future work.

Note on AI Use

Over the course of the project, AI has been used for assistance in certain areas and foregone in others, to best balance the quality of work and speed of progression. One of the key areas where AI was used was helping to brainstorm ideas for implementation approaches, and creating initial prototype code implementations for each stage. This significantly sped up the design process of the proposed solution, and allowed quick testing of the viability of new ideas within the code. AI was also used for additional development tasks such as setting up the development environment across different machines; assisting with the evaluation process by crafting edge cases for standards coverage/robustness testing; and searching for possible areas of regression with Flang implementation. Finally, AI was used during the literature review to help identify relevant papers and summarise key points from papers that had partial relevance.

All of the paper write-up and diagrams were written/created without the use of AI to ensure correctness and get the points across in the most succinct manner. The solution design process was supplemented throughout by manual review and cross-referencing with literature and documentation, and all code has been manually reviewed, modified, and significantly refined to improve the implementation as well as its clarity and maintainability.

References

- [1] G.R. Mudalige, M.B. Giles, I. Reguly, C. Bertolli, and P.H.J. Kelly. Op2: An active library framework for solving unstructured mesh-based applications on multi-core and many-core architectures. In *2012 Innovative Parallel Computing (InPar)*, pages 1–12, 2012.
- [2] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [3] M. Giles, Devendra Ghate, and Mihai Duta. Using automatic differentiation for adjoint cfd code development. *Computational Fluid Dynamics Journal*, 16, January 2008.
- [4] Carlo Bertolli, Adam Betts, Gihan R. Mudalige, Mike B. Giles, and Paul H.J. Kelly. Design and performance of the OP2 library for unstructured mesh applications. In *Euro-Par 2012: Parallel Processing Workshops*, volume 7640 of *Lecture Notes in Computer Science*, pages 191–200. Springer, 2013.
- [5] Mohamed Aissa. *GPU-accelerated CFD Simulations for Turbomachinery Design Optimization*. PhD thesis, October 2017.
- [6] I. Z. Reguly, D. Gopinathan, J. H. Beck, M. B. Giles, S. Guillas, and F. Dias. The Volna-OP2 tsunami code (version 1.0). *Geoscientific Model Development Discussions*, 2018.
- [7] N. Jacobsen. LLVM supported source-to-source translation: Translation from annotated C/C++ to CUDA C/C++. Master’s thesis, University of Oslo, 2016.
- [8] M.B. Giles, G.R. Mudalige, Z. Sharif, G. Markall, and P.H.J. Kelly. Performance analysis and optimization of the op2 framework on many-core architectures. *The Computer Journal*, 55(2):168–180, 2012.
- [9] Andrew Porter, Rupert Ford, Balthasar Reuter, Joerg Henrichs, and Pearu Peterson. fparser2: A fortran parser, 2021. <https://fparser.readthedocs.io/en/latest/fparser2.html>.
- [10] The LLVM Project. “llvm flang”, 2026. <https://flang.llvm.org/docs/FortranStandardsSupport.html>.
- [11] G.R. Mudalige, M.B. Giles, J. Thiyaalingam, I.Z. Reguly, C. Bertolli, P.H.J. Kelly, and A.E. Trefethen. Design and initial performance of a high-level unstructured mesh framework on heterogeneous parallel systems. *Parallel Computing*, 39(11):669–692, 2013.
- [12] M.B. Giles, G.R. Mudalige, B. Spencer, C. Bertolli, and I. Reguly. Designing op2 for gpu architectures. *Journal of Parallel and Distributed Computing*, 73(11):1451–1460, 2013. Novel architectures for high-performance computing.
- [13] G.D. Balogh, G.R. Mudalige, I.Z. Reguly, S.F. Antao, and C. Bertolli. Op2-clang: A source-to-source translator using clang/llvm libtooling. In *2018 IEEE/ACM 5th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, pages 59–70, 2018.
- [14] I. Z. Reguly, G. R. Mudalige, M. B. Giles, D. Curran, and S. McIntosh-Smith. The OPS domain specific abstraction for multi-block structured grid computations. In *Proceedings of the 2014 Fourth International Workshop on Domain-Specific Languages and High-Level Frameworks for High Performance Computing, WOLFHPC ’14*, pages 58–67. IEEE Computer Society, 2014.

- [15] Florian Rathgeber, David A. Ham, Lawrence Mitchell, Michael Lange, Fabio Luporini, Andrew T. T. Mcrae, Gheorghe-Teodor Borcea, Graham R. Markall, and Paul H. J. Kelly. Firedrake: Automating the finite element method by composing abstractions. *ACM Transactions on Mathematical Software*, 43:24:1–24:27, December 2016.
- [16] Christian T. Jacobs, Satya P. Jammy, and Neil D. Sandham. OpenSBLI: A framework for the automated derivation and parallel execution of finite difference solvers on a range of computer architectures. *Journal of Computational Science*, 18:12–23, 2017.
- [17] Michael Lange, Navjot Kukreja, Mathias Louboutin, Fabio Luporini, Felipe Vieira, Vincenzo Pandolfo, Paulius Velesko, Paulius Kazakas, and Gerard Gorman. Devito: Towards a generic finite difference DSL using symbolic Python. In *Proceedings of the 6th Workshop on Python for High-Performance and Scientific Computing, PyHPC '16*, pages 67–75, Piscataway, NJ, USA, 2016. IEEE Press.
- [18] Eli Bendersky. Modern source-to-source transformation with Clang and LibTooling, 2014. <https://eli.thegreenplace.net/2014/07/29/ast-matchers-and-clang-refactoring-tools>.
- [19] Jingyue Wu, Artem Belevich, Eli Bendersky, Mark Heffernan, Chris Leary, Jacques Pienaar, Bjarke Roune, Rob Springer, Xuetian Weng, and Robert Hundt. gpucc: An open-source GPGPU compiler. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, CGO '16, pages 105–116. ACM, 2016.
- [20] Matteo Marangoni and Thomas Wischgoll. ToGPU: Automatic source transformation from C++ to CUDA using Clang/LLVM. *Electronic Imaging*, 2016(1):1–9, 2016.
- [21] C. Lattner and V. Adve. Llvm: a compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization, 2004. CGO 2004.*, pages 75–86, 2004.
- [22] Güray Özen, Simone Atzeni, Michael Wolfe, Annemarie Southwell, and Gary Klimowicz. Openmp gpu offload in flang and llvm. In *2018 IEEE/ACM 5th Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, pages 1–9, 2018.
- [23] Gabriel Rodriguez-Canal, David Katz, and Nick Brown. An mlir pipeline for offloading fortran to fpgas via openmp. In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1096–1103. ACM, November 2025.
- [24] Eric Wright, Johannes Doerfert, Shilei Tian, Barbara Chapman, and Sunita Chandrasekaran. Implementing openmp’s simd directive in llvm’s gpu runtime. In *Proceedings of the 52nd International Conference on Parallel Processing, ICPP '23*, pages 173–182, New York, NY, USA, 2023. Association for Computing Machinery.
- [25] LFortran Contributors. LFortran, 2026. <https://github.com/lfortran/lfortran>.
- [26] Tree-sitter Contributors. Tree-sitter, 2026. Accessed 2026.
- [27] Matt Stadelman and contributors. tree-sitter-fortran: A fortran grammar for tree-sitter, 2026.
- [28] Olaf Krzikalla, Konrad Feldhoff, Ralph Müller-Pfefferkorn, and Wolfgang E. Nagel. Scout: A source-to-source transformator for SIMD-optimizations. In *Proceedings of the 2011 International Conference on Parallel Processing – Volume 2, Euro-Par'11*, pages 137–145, Berlin, Heidelberg, 2012. Springer-Verlag.

- [29] Seyong Lee, Seung-Jai Min, and Rudolf Eigenmann. OpenMP to GPGPU: A compiler framework for automatic translation and optimization. *ACM SIGPLAN Notices*, 44(4):101–110, 2009.
- [30] The LLVM Project. Flang Parsing Documentation, 2026. <https://github.com/llvm/llvm-project/blob/main/flang/docs/Parsing.md>.
- [31] The LLVM Project. [flang] Add traits to more AST nodes. LLVM Project pull request 175566, 2026. <https://github.com/llvm/llvm-project/pull/175566>.
- [32] The LLVM Project. [flang] Add traits to more AST nodes (accessors). LLVM Project pull request 175578, 2026. <https://github.com/llvm/llvm-project/pull/175578>.
- [33] Canonical Ltd. Ubuntu 24.04 LTS (Noble) libflang packages, 2026. <https://packages.ubuntu.com/search?keywords=libflang&searchon=names&suite=noble§ion=all>.
- [34] Gihan Mudalige, Istvan Reguly, and Mike Giles. OP2 documentation, 2026. https://op2-dsl.readthedocs.io/en/latest/getting_started.html.
- [35] Dimitri van Heesch. *Doxygen: Source Code Documentation Generator Tool*, 1997. Available at <https://www.doxygen.nl/>.
- [36] The Clang Team. ClangFormat, 2026. <https://clang.llvm.org/docs/ClangFormat.html>.
- [37] The LLVM Project. Debian and Ubuntu nightly packages, 2026. <https://apt.llvm.org/>.
- [38] Samuel Armstrong. LLVM Flang integration as an alternative Fortran translation path. OP2-Common pull request 267, 2026. <https://github.com/OP-DSL/OP2-Common/pull/267>.